Tab 1

Final Report

Pantry Inventory Management System

Andrew Feran
Arselle Conteh
Benjamin Bruinsma
Davi Borges
Nickolas Diaz

Florida Polytechnic University
June 15, 2025

Table of Contents

Software Requirements Specification

1. Introduction

- 1.1 Purpose
- 1.2 Intended Audience and Reading Suggestions
- 1.3 Product Scope

2. Overall Description

- 2.1 Product Perspective
- 2.2 Product Functions
  - 2.2.1 Core features
  - 2.2.2 Basic Security Measures
- 2.3 User Classes and Characteristics
- 2.4 Operating Environment
- 2.5 Design and Implementation Constraints
- 2.6 User Documentation
- 2.7 Assumptions and Dependencies

3. External Interface Requirements

- 3.1 Hardware Interfaces
- 3.2 Software Interfaces
- 3.3 Communications Interfaces

4. System Features

- 4.1 Private and Protected Inventory
- 4.2 Item Management System
- 4.3 Pantry Search
- 4.4 Shopping List Creation
- 4.5 Remainder of Expiration Date
- 4.6 Basic Security Measures

5. Other Nonfunctional Requirements

- 5.1 Performance Requirements
- 5.2 Safety Requirements
- 5.3 Security Requirements
- 5.4 Software Quality Attributes

6. Other Requirements

- 6.1 Database Requirements

7. Use Case Model

- 7.1 Use Case Diagram
- 7.2 Use Case Descriptions

System Terms

Software Design Specification

1. Introduction

1.1 Purpose

1.2 System Overview

2. Design Considerations

2.1 Assumptions

2.2 General Constraints

3. System Architecture

4. Detailed System Design

4.1 Class Diagram

4.2 Activity Diagram

4.3 Sequence Diagram

4.4 State Transition Diagram

4.5 Algorithms for Components/Methods

4.5.1 Expiration Alert System

4.5.2 User Authentication

4.5.3 Auto-Generate Shopping List

4.5.4 Shopping List Purchase Processing

4.6 Database Design

Users

Pantry Items

Notifications

Shopping Lists

Shopping List Items

5. User Interface Design

Test cases and testing results

Pantry Management System Test Cases 1. Database Schema Validation

2.1 User-Pantry Item Relationship

2.2 Negative Quantity Prevention

3.1 Expiration Notification Generation

3.2 Notification Read Status

4.1 List-Item Association

4.2 Purchase Status Update

5.1 Email Uniqueness Enforcement

5.2 Role Validation

6.1 Index Efficiency

7.1 Password Hashing

7.2 SQL Injection Prevention

Summary of Test Results

Screenshots of implemented software solution

Lessons learned for the project

Software Requirements Specification

for

Pantry Inventory Management System

Version 1.8

Prepared by Andrew Feran, Arselle Conteh, Benjamin Bruinsma, Davi Borges, Nickolas Diaz

Florida Polytechnic University

06/01/2025

Revision History

Name	Date	Reason For Changes	Version
	5/30/25	Init. Ver.	1.0
Andrew Feran	5/31/25	Completed Sec 5. + 6.1	1.1
Nickolas Diaz	5/31/25	Completed Sec. 1, + 2.2 added to 5.3	1.2
Arselle Conteh	5/31/25	Completed Sec. 3, +2.4	1.3
Nickolas Diaz	6/1/25	Completed section 2.1 and 2.5	1.4
Davi Borges	6/1/25	Completed Sec 4.	1.5
Davi Borges	6/1/25	Adjusted page numbering and formatting.	1.6
Andrew Feran	6/1/25	Small tweaks + Glossary	1.7
Andrew Feran	6/1/25	Section 7	1.8

1. Introduction

1.1 Purpose

The pantry management system is a web app that allows users to manage their own pantry systems, which includes adding, modifying and deleting items as well as adding expiration dates. Users can also create shopping lists for missing items and have remainders for their products. This app is meant for users/businesses to manage their own businesses and be able to keep track of all their items. Website Link GitHub Source Code

1.2 Intended Audience and Reading Suggestions

Developers: sections 1-7 Know the application to the front and back and develop new features and maintain the software.
Marketing staff: sections 1,2,4 Know what the product is and market the features
Users: sections 1,2 Be able to use the software
Testers: sections 1,2,4,5 Know what the product features are and how to test and break it.
Documentation writers: sections 1,2 Write documentation for all the different types of readers
Stockholders/CEO: 1-4 Know the application and be able to manage the business
Project managers: sections 1-7 know the whole project and be able to manage devs to develop and maintain, be able to explain the team's work to the Stockholders/CEO. Make the marketing staff market the material. Make testers test the software and manage the documentation of it.

1.3 Product Scope

The purpose of the pantry inventory management system is to make it easier for businesses and employees to manage their pantry. Employees will be able to better manage their items and users will be able to search and purchase items more easily. The goal of this business is to be able to provide as much help to users regarding managing their pantry.

2. Overall Description

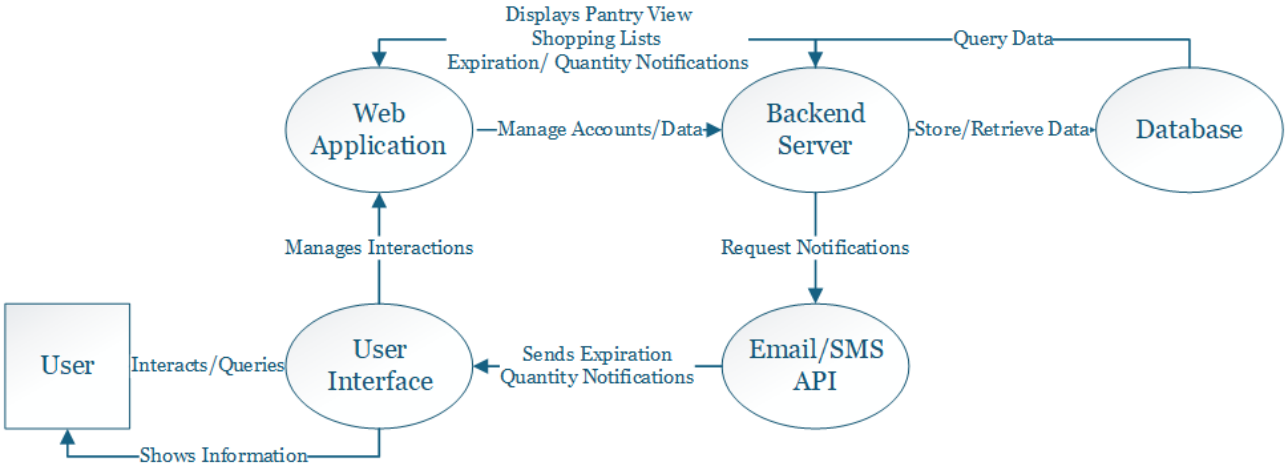
2.1 Product Perspective

The pantry inventory management system is a new self-contained product. It is meant to solve the problem of businesses/households keeping track of their pantry. It is meant to help people manage their inventory efficiently and cleanly and to keep track/notify the quantity of items and expiration dates. The product is aimed for everyone of all sizes, it is simple enough for families and full of features for large businesses.

Components of the system:

user interface through the web
web application
backend server
database
user/manager
notification (in-app, or SMS, email)
Third party services/api

Diagram:



2.2 Product Functions

2.2.1 Core features

Private and protected inventory: by providing user authentication to the staff and users
Item management system: for pantry items which allows users to add, update, and remove items with information such as names, quantities, and expiration dates
Pantry search: allows users to search items according to quantity, name, price and expiration date
Shopping list creation: users can create shopping lists, and buy items from that list
Remainders of expiration date: Users will be notified if their items are nearing their expiration date, possibly via push/email or highlighting.

2.2.2 Basic Security Measures

Input validation: All input into the server must be sanitised before being processed
Role-based access control: Reading and editing pantries are only given to the users
Session: Once logged in users must remain logged in during their session, using Cookies-Based Authentication, token authentication through (JWT).

2.3 User Classes and Characteristics

User Class	Frequency of Use	Technical Expertise	Notes
Regular User	Medium	Low	Class represents customers who use the product
System Admins	Medium	High	Maintain system consistency and usability
Developers and Testing	High	High	Use the system during testing and development

- **Regular Users:**
  - Description: Everyday users who use the application to manage their panty items
  - Use the system: Daily or Weekly
  - Importance: High, this system is designed to accommodate these users. This user's satisfaction is the most important user of the system.
  - Technical Expertise: Low, basic computer skills
- **System Admins:**
  - Description: The people in charge of managing the system. They are in charge of maintaining the system and troubleshooting system errors.
  - Use the system: As needed when issues surface
  - Importance: Medium, a degree of usability is required but unnecessary to place effort into
  - Technical Expertise: Advanced, experience in information technology is required
- **Developers:**
  - Description: The team of developers who build the system
  - Use the system: High during testing and development
  - Importance: Medium, only relevant during development
  - Technical Expertise: High, experience in software development

2.4 Operating Environment

The environment for this product is a web-based application that is designed to operate within a standard client-server architecture. The client side will need to be able to run more modern web browsers (such as Safari, Google Chrome, and Edge), and it needs to support HTML5, CSS3, and JavaScript (ES6). As for the server side, a Linux-based environment will likely need to be integrated with an HTTP server (something similar to Ubuntu Server 22.04 LTS/Apache HTTP Server 2.4) which will act as the host. We will use MySQL to run the database for this system due to its data managing capabilities. The system needs to be peacefully integrated with the email service, as well as the security software.

2.5 Design and Implementation Constraints

<Describe any items or issues that will limit the options available to the developers. These might include: corporate or regulatory policies; hardware limitations (timing requirements, memory requirements); interfaces to other applications; specific technologies, tools, and databases to be used; parallel operations; language requirements; communications protocols; security

considerations; design conventions or programming standards (for example, if the customer's organization will be responsible for maintaining the delivered software).>

Major constraint is finding a place to host the system whether it is a hosting service or personal hardware. We need to be able to deploy a python webserver, Mysql database and potential api's for the notifications.

Python web server: It needs to process data coming into and out of the database. It will need to manage all the functions for the entire system. It needs a secure connection to the MySQL database, any APIs, and to the client. It also has to manage users throughout their session for secure access.

MySQL database: The schema design needs to be able to store all the information needed, SQL queries to modify, add, delete and query the data. Data needs to be processed before entering the database. If the schema changes so does the way it is accessed. Performance tuning: tables need to be indexed and have good connections to other tables.

User interface: It has to be user friendly and show the necessary information, and actions through buttons written in HTML and CSS. It also has to support the logging and logging out of users.

Web interface: It has to connect the client interfaces to the web server to perform the actions of the users. It has to manage all the cookies used and it will be written in javascript.

API: It will need to be configured to send notifications to email addresses.

Security: The web app needs secure password storage and session management, it needs good role access control for the lists and inventory and it needs good api security

Mitigating Errors: error responses should not be sent back to the client. Errors needs to be wrapped within try catch blocks so if one happens the web app won't break

Documentation: If a error or a new feature is needed modification to the documentation is needed

2.6 User Documentation

- User manuals: PDF documentation
    - An in-depth analysis of the system functions and how to use them
    - Specific help can be found in this document for most problems
- On-line help:
    - A small button marked '?' with specific help when the user is stuck
    - A FAQ page of issues users have and solutions to their problem
- Tutorials: Quick-start tutorial
    - A quick presentation of mainly images showing the basic functions the user can use in the system

- A short video showcasing the various functionality of the application

2.7 Assumptions and Dependencies

We assume that:

- There are no more than 500 items in the pantry at a time
- There are no more than 10 concurrent users
- The HTTPS protocol is secure
- The probability of collision of hashed passwords is zero due to cryptographic primitives
- Users may access this site from any number of web supported device types

This system will inherently rely on the security and reliability of:

- Python 3 as the codebase
- Flask as the backend of the web application
- MySQL as the database
- Apache2 as the web server
- Ubuntu as the host OS

3. External Interface Requirements

3.1 Hardware Interfaces

Some of the physical characteristics of this interface consist of the User Devices, such as the physical PC, tablet, smart phones, or anywhere that this interface is being accessed from. The backend server (the web server) is one of the main logical characteristics of the interface and its main purpose would be handling requests or processing data. Other logical characteristics would consist of the database (that takes care of the items and queries) as well as a potential notification system that keeps all of the users aware of all the changes.

3.2 Software Interfaces

Considering the scale of this project, there are multiple components to consider, and a lot of elements are subject to change. For the database, we're most likely using MySQL 8.0 for the construction process. This ensures that all of the pantry-related data is stored and managed properly. The OS will most likely be Linux-based considering how uniform the structure is. The main purpose would be to host the web/database servers, and background task schedulers. The main point of the web servers would be to handle the HTTP requests from the users and link them back to the code. The backend Applications would handle the business logic layers which process the pantry actions and interact with the database. Considering how we would still implement the notification system, the main action this service provides consists of sending emails that alert the users about any stock updates. The implementation process consists of experimenting with various types of code and formatting to ensure that this process flows smoothly.

3.3 Communications Interfaces

One of the requirements that we'll use with this product is a modern browser that has HTTP support, with the main goal of interacting with the code and the encryption. The email's main functionality would be to notify any user about restocking when the items are close to their expiration or low. We can also have standards which aim to uniform the whole product (like accepting HTTPS, SMTP, HTTP, APIs, and standards along those lines). All web communications are encrypted and session data should be protected with some form of cookies. The messaging format should include the subject ([Pantry Alert] item(s) Expiring Soon), a greeting (Hello [User],), and a basic interface that shows all of the items and their expiration dates.

4. System Features

This section details the major services and functionalities provided by the Pantry Inventory Management System. Each feature is described in terms of its purpose, priority, user interactions (stimulus/response sequences), and specific functional requirements.

4.1 Private and Protected Inventory

4.1.1 Description and Priority

This feature will provide the user the authentication process, that will protect both staff and general users. Making sure the access to the pantry inventory is private and protected from unauthorized access.
Priority: *HIGH*

4.1.2 Stimulus/Response Sequences

User Action: *User navigates to the login/signup page.*
System Response: *System displays the login and signup forms.*

User Action: *Enter a valid signup information. (Email and password.)*
System Response: *System creates a new account with the provided information and logs the user in.*

User Action: *User enters valid login information.*
System Response: *System authenticates the user and grants access to their personalized pantry.*

User Action: *Enter an invalid login information.*
System Response: *System displays a message informing that the information entered is incorrect ("Invalid email or password").*

4.1.3 Functional Requirements

REQ-1: The system will allow users to register for a new account by providing a unique email and password.
REQ-2: The system will allow users to log in to their account using their registered email and password.

REQ-3: The system will require users to create a password with a minimum length and a combination of uppercase letters, lowercase letters, and numbers.

4.2 Item Management System

4.2.1 Description and Priority

This feature allows users to manage their pantry items. They can add new items, update existing ones, and remove items. This includes details like names, quantities, and expiration dates.
Priority: *HIGH*

4.2.2 Stimulus/Response Sequences

User Action: *User logs into their account.*
System Response: *System displays the user's pantry inventory.*

User Action: *User selects to add a new item.*
System Response: *System displays a form to enter item details.*

User Action: *User enters item details (name, quantity, expiration date) and saves.*
System Response: *System adds the new item to the pantry and updates the display.*

User Action: *User selects an item to update.*
System Response: *System displays the item's details in an editable form.*

User Action: *User modifies item details and saves.*
System Response: *System updates the item's information and refreshes the display.*

User Action: *User selects an item to remove.*
System Response: *System prompts for confirmation.*

User Action: *User confirms removal.*
System Response: *System deletes the item from the pantry.*

4.2.3 Functional Requirements

REQ-1: The system will allow authenticated users to add new pantry items with name, quantity, and expiration date.
REQ-2: The system will allow authenticated users to update details of existing pantry items.
REQ-3: The system will allow authenticated users to remove pantry items.

4.3 Pantry Search

4.3.1 Description and Priority

This feature lets users search for items in their pantry based on criteria like quantity, name, price, and expiration date. It helps users quickly find specific items.
Priority: *MEDIUM*

4.3.2 Stimulus/Response Sequences

User Action: *User views their pantry inventory.*
System Response: *System displays all pantry items.*

User Action: *User enters a search query (item name, quantity, expiration date).*
System Response: *System filters and displays matching pantry items.*

User Action: *User clears the search query.*
System Response: *System displays all pantry items again.*

4.3.3 Functional Requirements

REQ-1: The system will provide a search function for pantry items.
REQ-2: Users will be able to search for items by name.
REQ-3: Users will be able to search for items by quantity.
REQ-4: Users will be able to search for items by expiration date.

4.4 Shopping List Creation

4.4.1 Description and Priority

This feature allows users to create and manage shopping lists. Users can sort items based on low stock or expiring items. They can also mark items as bought.
Priority: *MEDIUM*

4.4.2 Stimulus/Response Sequences

User Action: *User navigates to the shopping list section.*
System Response: *System displays existing shopping lists and an option to create a new one.*

User Action: *User creates a new shopping list.*
System Response: *System creates and displays an empty shopping list.*

User Action: *User adds an item to a shopping list.*
System Response: *System adds the item to the list.*

User Action: *User selects to sort items on the shopping list.*
System Response: *System reorders items based on selected criteria (low stock, expiration date)*

User Action: *User marks a list as bought.*
System Response: *System updates the list status.*

4.4.3 Functional Requirements

REQ-1: The system will allow the user to create multiple shopping lists.
REQ-2: The system will allow the user to add items to a shopping list.
REQ-3: The system will allow the user to mark a list as “purchased”.
REQ-4: The system will allow users to sort shopping list items by criteria such as low stock or nearing expiration.

4.5 Remainder of Expiration Date

4.5.1 Description and Priority

This feature notifies users when their pantry items are nearing their expiration date. Notifications will be sent by email or SMS.
Priority: *HIGH*

4.5.2 Stimulus/Response Sequences

System Action: *System automatically checks for expiring items.*
System Response: *System triggers a notification process if items are nearing expiration.*

User Action: *User views their pantry inventory.*
System Response: *Expiring items are visually highlighted in the display.*

4.5.3 Functional Requirements

REQ-1: The system will automatically identify pantry items within a configurable expiration threshold.
REQ-2: The system will visually highlight expiring items in the pantry display.
REQ-3: The system will send notifications to users about expiring items via email or SMS.
REQ-4: Notifications will clearly state the expiring item(s) and their dates.

4.6 Basic Security Measures

4.6.1 Description and Priority

This feature ensures the integrity and protection of user data through input validation, role-based access control, and secure session management.
Priority: *HIGH*

4.6.2 Stimulus/Response Sequences

User Action: *User submits any data to the server (login, item details).*
System Response: *System sanitizes the input before processing.*

User Action: *User attempts to read or edit another user's pantry.*
System Response: *System denies access, as reading and editing pantries are restricted to the owner.*

User Action: *User successfully logs in.*
System Response: *System establishes a secure session using Cookies-Based Authentication or token authentication (JWT), keeping the user logged in during their session.*

4.6.3 Functional Requirements

REQ-1: All input into the server must be sanitized before being processed to prevent vulnerabilities.
REQ-2: Role-based access control will be implemented, ensuring that reading and editing of pantries are only permitted to the respective users.
REQ-3: Once logged in, users must remain logged in securely during their session, utilizing either Cookies-Based Authentication or token authentication through JWT

5. Other Nonfunctional Requirements

5.1 Performance Requirements

P.R-1 The system shall be responsive (< 1000 ms) to all queries with the assumption that the pantry holds no more than 500 items.

P.R-2 The system shall be responsive (< 1000 ms) under load with the assumption that there will be no more than 10 concurrent users.

5.2 Safety Requirements

Saf.R-1 The system shall track the expiration dates of all products in the pantry and dispose of them automatically.

5.3 Security Requirements

Sec.R-1 The software shall enforce strong authentication requiring high entropy passwords

Sec.R-2 All passwords must be hashed by either bcrypt, Argon2, scrypt, SHA-512 or PBKDF2 with a salt to prevent any rainbow/dictionary attacks.

Sec.R-3 All user input must be sanitized before being sent/processed by the server to prevent XSS or SQL injection attacks

Sec.R-4 User authentication must use cookies-based authentication (needs csrf tokens), or token authentication through (JWT)

5.4 Software Quality Attributes

Accessibility: The frontend shall be primarily designed for desktop web browsers and shall follow HTML 5 best practice for accessibility.

Availability: The application shall be accessible on demand via startup script.

Data Integrity: The database shall follow data storage best practices for data integrity. More in sec. 6.1

6. Other Requirements

6.1 Database Requirements

Db.R-1 The system must use a relational database (e.g., PostgreSQL) to ensure data consistency.

Db.R-2 All personally identifiable information (PII) must be encrypted at rest and in transit.

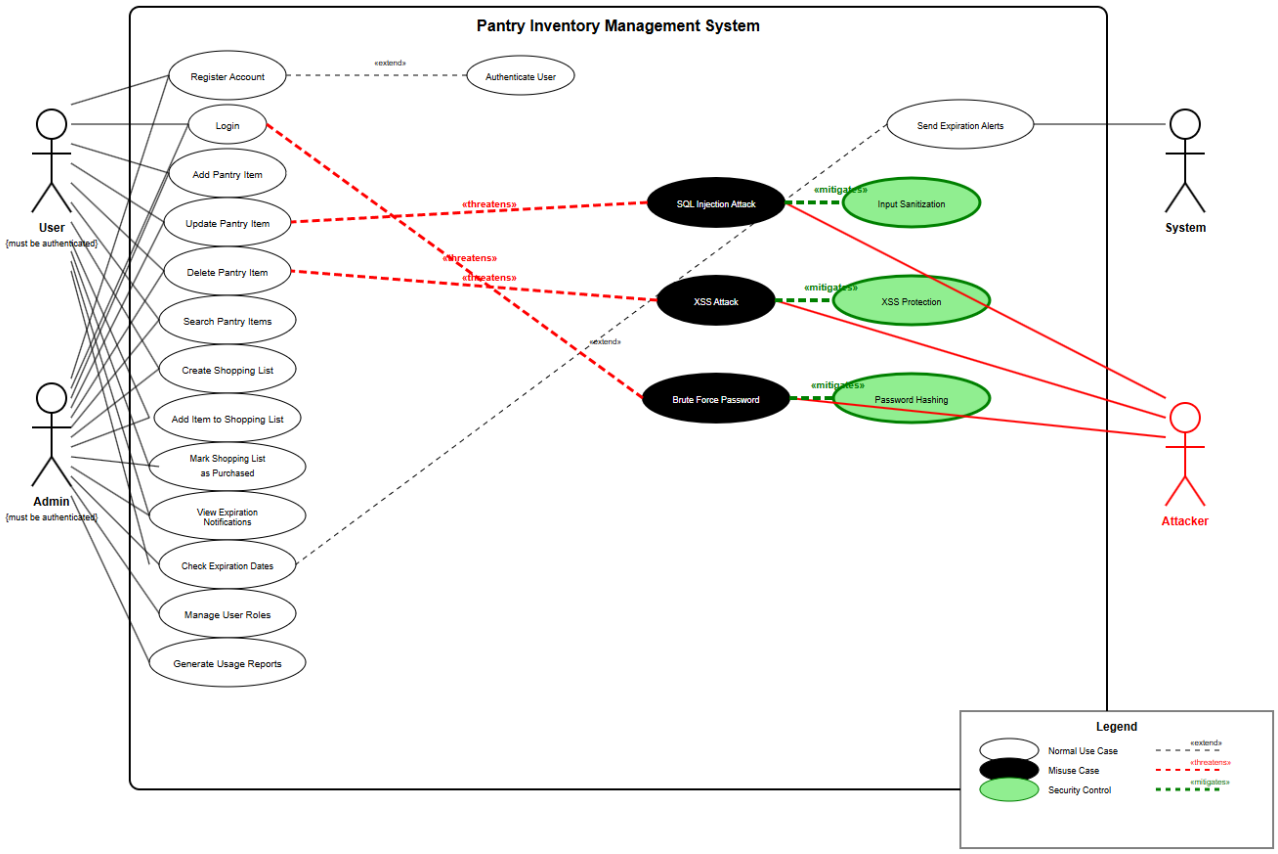
Db.R-3 The database should support daily automated backups with a 7-day retention period.

Db.R-4 The system must maintain referential integrity between related tables using foreign key constraints.

7. Use Case Model

7.1 Use Case Diagram

For clarification, users/admins **must** be authenticated for any and all use cases shown in the following diagram.



7.2 Use Case Descriptions

Use Case Number	UC-001
Use Case Name	Register Account
Author/Source	Andrew Feran
Date of Creation	6/1/2025
Precondition(s)	User is not logged in
Successful Post Condition	User is registered and logged into a new account
Actors	User, Admin
Priority	High
Related Use Cases	Login, Logout

Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Click register	SR1	Return registration page
UA2	Enter Email and password and submit	SR2	Register account and login user
Alternative I: Invalid information			
User Action		**System Response**	
A1.UA1	Click register	A1.SR1	Return registration page
A1.UA2	Enter email and password and submit	A1.SR2	Display error message and do not proceed

Use Case Number	UC-002

Use Case Name	Login
Author/Source	Andrew Feran
Date of Creation	6/1/2025
Precondition(s)	User is registered but not logged in
Successful Post Condition	User is registered and logged into existing account
Actors	User, Admin
Priority	High
Related Use Cases	Register, Logout

Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Click login	SR1	Return login page
UA2	Enter valid email/password and submit	SR2	Successfully authenticate user
Alternative I: Non-existent Account			
User Action		**System Response**	
A1.UA1	Click login	A1.SR1	Return login page
A1.UA2	Enter non-existent email	A1.SR2	Inform user that the account does not exist
Alternative 2: Incorrect Password			
User Action		**System Response**	
A2.UA1	Click login	A2.SR1	Return login page
A2.UA2	Enter wrong password	A2.SR2	Inform user that the password is incorrect

Use Case Number	UC-003
Use Case Name	Logout
Author/Source	Andrew Feran
Date of Creation	6/1/2025
Precondition(s)	User is currently logged in
Successful Post Condition	User is logged out
Actors	User, Admin
Priority	Medium
Related Use Cases	Register, Login

Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Click logout	SR1	Ask the user to confirm
UA2	Click Confirm	SR2	Successfully logout user
Alternative I: Cancel			
User Action		**System Response**	
A1.UA1	Click logout	A1.SR1	Ask the user to confirm
A1.UA2	Click Cancel	A1.SR2	Cancel logout process

Use Case Number	UC-004
Use Case Name	Add item to pantry
Author/Source	Andrew Feran
Date of Creation	6/1/2025
Precondition(s)	User is logged in and viewing the inventory page
Successful Post Condition	Item is in pantry or item quantity increment accordingly

Actors		User, Admin	
Priority		High	
Related Use Cases		Update Pantry Item, Delete Pantry Item, Search Pantry Item	
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Click add item	SR1	Display a form in a modal
UA2	Enter name, qty, and exp. Date and submit	SR2	Successfully add item to pantry
Alternative I: Invalid item			
User Action		**System Response**	
A1.UA1	Click add item	A1.SR1	Display a form in a modal
A1.UA2	Enter invalid name, qty, or exp. Date and submit	A1.SR2	Inform user that the item couldn't be added

Use Case Number		UC-005	
Use Case Name		Update item in pantry	
Author/Source		Andrew Feran	
Date of Creation		6/1/2025	
Precondition(s)		User is logged in and viewing the inventory page	
Successful Post Condition		Item details are updated according to the user's request	
Actors		User, Admin	
Priority		High	
Related Use Cases		Add Pantry Item, Delete Pantry Item, Search Pantry Item	
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Click update item	SR1	Display a form in a modal
UA2	Enter name, qty, exp. Date and submit	SR2	Successfully update item in pantry
Alternative I: Invalid information			
User Action		**System Response**	
A1.UA1	Click update item	A1.SR1	Display a form in a modal
A1.UA2	Enter invalid name, qty, or exp. Date and submit	A1.SR2	Inform user that the item couldn't be updated

Use Case Number		UC-006	
Use Case Name		Delete item in pantry	
Author/Source		Andrew Feran	
Date of Creation		6/1/2025	
Precondition(s)		User is logged in and viewing the inventory page	
Successful Post Condition		Item is deleted or count is decremented accordingly	
Actors		User, Admin	
Priority		High	
Related Use Cases		Add Pantry Item, Update Pantry Item, Search Pantry Item	
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Click delete item	SR1	Display a confirmation alert

UA2	Confirm	SR2	Successfully remove item from pantry
Alternative I: Cancel			
User Action		**System Response**	
A1.UA1	Click delete item	A1.SR1	Display a confirmation alert
A1.UA2	Cancel	A1.SR2	Do not remove the item from pantry

Use Case Number	UC-007		
Use Case Name	Search items in pantry		
Author/Source	Andrew Feran		
Date of Creation	6/1/2025		
Precondition(s)	User is logged in and viewing the inventory page		
Successful Post Condition	Relevant items based on criteria are returned		
Actors	User, Admin		
Priority	High		
Related Use Cases	Add Pantry Item, Update Pantry Item, Delete Pantry Item		
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Enter information to the search form & search	SR1	Query the database and return relevant items
Alternative I: Search All			
User Action		**System Response**	
A1.UA1	Enter no information to the search form & search	A1.SR1	Query database and return all items
Alternative 2: Item(s) not found			
User Action		**System Response**	
A2.UA1	Enter information to the search form & search	A2.SR1	Query the database and respond “no items…”
Alternative 3: Search All			
User Action		**System Response**	
A3.UA1	Enter no information to the search form & search	A3.SR1	Query the database and respond “no items…”

Use Case Number	UC-009		
Use Case Name	Add item to list		
Author/Source	Andrew Feran		
Date of Creation	6/1/2025		
Precondition(s)	User is logged in and viewing the inventory page		
Successful Post Condition	Shopping list is updated per user’s request		
Actors	User, Admin		
Priority	Medium		
Related Use Cases	Create shopping list, Mark list as purchased		
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Click add to list	SR1	Display a confirmation
UA2	Click confirm	SR2	Successfully add item to list
Alternative I: Cancel			
User Action		**System Response**	
A1.UA1	Click add to list	A1.SR1	Display a confirmation

A1.UA2	Click Cancel	A1.SR2	Cancel

Use Case Number	UC-010		
Use Case Name	Mark list as purchased		
Author/Source	Andrew Feran		
Date of Creation	6/1/2025		
Precondition(s)	User is logged in and viewing a shopping list		
Successful Post Condition	Shopping list deleted and inventory updated accordingly		
Actors	User, Admin		
Priority	Medium		
Related Use Cases	Add item to list, Create shopping list		
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Click “mark as purchased”	SR1	Display a confirmation
UA2	Click confirm	SR2	Delete list, update inventory accordingly
Alternative I: Cancel			
User Action		**System Response**	
A1.UA1	Click “mark as purchased”	A1.SR1	Display a confirmation
A1.UA2	Click Cancel	A1.SR2	Cancel

Use Case Number	UC-011		
Use Case Name	View expiration notifications		
Author/Source	Andrew Feran		
Date of Creation	6/1/2025		
Precondition(s)	User is logged in and viewing the notifications tab		
Successful Post Condition	User can view ANY and ALL expired items in the pantry		
Actors	User, Admin		
Priority	Urgent		
Related Use Cases	Check expiration dates, Send expiration alerts		
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Click notifications tab	SR1	Show notifications
Alternative I: No notifications to display			
User Action		**System Response**	
A1.UA1	Click notifications tab	A1.SR1	Show message “No expiration notifications”

Use Case Number	UC-012
Use Case Name	Check expiration dates
Author/Source	Andrew Feran
Date of Creation	6/1/2025
Precondition(s)	User is logged in and viewing the inventory page
Successful Post Condition	User can determine the expiration dates of products

Actors		User, Admin	
Priority		Urgent	
Related Use Cases		View expiration notifications, Send expiration alerts	
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	User clicks and item's information	SR1	Modal pops up
UA2	User reads information and hits close	SR2	Modal closes

Use Case Number		UC-013	
Use Case Name		Manage user roles	
Author/Source		Andrew Feran	
Date of Creation		6/1/2025	
Precondition(s)		User is authenticated with admin permissions	
Successful Post Condition		Administrator manages roles of other users	
Actors		Admin	
Priority		High	
Related Use Cases			
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Navigate to admin area	SR1	Display admin area
UA2	Click to users page	SR2	Display users page
UA3	Click user	SR3	Display modal
UA4	Enter user roles and confirm	SR4	Update user roles and close modal

Use Case Number		UC-014	
Use Case Name		Generate usage reports	
Author/Source		Andrew Feran	
Date of Creation		6/1/2025	
Precondition(s)		User is authenticated with admin permissions	
Successful Post Condition		Administrator can generate usage reports	
Actors		Admin	
Priority		Low	
Related Use Cases			
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Navigate to admin area	SR1	Display admin area
UA2	Click to usage page	SR2	Display usage page
UA3	Click to export usage to csv	SR3	Compile usage to CSV and upload

Use Case Number	UC-015
Use Case Name	Send expiration alerts

Author/Source	Andrew Feran		
Date of Creation	6/1/2025		
Precondition(s)	There are expiration alerts that need to be sent		
Successful Post Condition	SMTP receipt is confirmed on the alerts		
Actors	User		
Priority	Urgent		
Related Use Cases	Check expiration dates, View expiration notifications		
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Has expired items in pantry	SR1	Send email alert
UA2	SMTP server responds with received	SR2	Success!
Alternative I: SMTP not received			
User Action		**System Response**	
A1.UA1	Has expired items in pantry	A1.SR1	Send email alert
A1.UA2	SMTP server responds with failed	A1.SR2	Log issue for Sys. Admins

Appendix A: Glossary

Authentication & Security Terms:

- **JSON Web Token (JWT):** Standard for securely transmitting information between parties as tokens
- **bcrypt:** Password hashing function designed to be slow and resistant to brute-force attacks
- **Argon2:** Modern password hashing function recommended for secure password storage
- **scrypt:** Password-based key derivation function designed to be memory-hard
- **Password-Based Key Derivation Function 2 (PBKDF2):** Password-Based Key Derivation Function used for password hashing
- **Secure Hash Algorithm 512 (SHA-512):** Cryptographic hash function producing 512-bit hash values
- **Salt:** Random data added to passwords before hashing to prevent rainbow table attacks
- **Cross-Site Request Forgery (CSRF) tokens:** Security tokens used to prevent Cross-Site Request Forgery attacks
- **Session-based authentication:** a stateful approach where a server maintains a record of a user's authenticated state for a specific period. Needs CSRF tokens to be secure
- **Stateless vs Stateful:** Stateless systems treat each request as a new, independent event, while stateful systems maintain information about past interactions to provide a more personalized or continuous experience.
- **Session management:** Process of maintaining user login state during application use
- **Rainbow attacks:** Cryptographic attack using precomputed tables of hash values to crack passwords
- **Dictionary attacks:** Password cracking method using lists of common passwords
- **Salt:** random data added to a password before it is hashed and stored protects against rainbow attacks
- **TLS (Transport Layer Security)**: a cryptographic protocol that provides privacy and data integrity for communications over the internet. It secures Protocols like https previously called SSL newest version is 1.3

- **Hashing vs Encryption:** Encryption protects data confidentiality by making it unreadable to unauthorized users, while hashing ensures data integrity and can be used for password storage. Hashing is irreversible, meaning it's computationally infeasible to recover the original data from the hash, whereas encryption is reversible with a key.
- **Sanitation:** process of cleaning and modifying user input to remove or neutralize any potentially harmful characters or code. Protects against SQL injection and XSS
- **Two-factor authentication (2FA)**: security measure that requires users to provide two separate forms of identification to verify their identity when logging into an account protects against dictionary attacks

Database Terms:

- **MySQL:** is an open-source relational database management system
- **PostgreSQL:** is an advanced open-source relational database system
- **Structured Query Language (SQL):** is a programming language for managing relational databases
- **Primary key:** a column (or a group of columns) that uniquely identifies each row in a table
- **Foreign key constraints:** Database rules that maintain referential integrity between tables
- **Referential integrity:** Database property ensuring relationships between tables remain consistent
- **Personally Identifiable Information (PII):** Data that can identify a specific individual
- **SQL injection:** an attack in which the attacker injects malicious SQL code into an application, allowing the attacker to view or modify a database

Web Technologies:

- **HyperText Transfer Protocol/HyperText Transfer Protocol Secure (HTTP/HTTPS):** are common communication protocols for web applications (secure version uses encryption)
- **HyperText Markup Language 5 (HTML5):** is the latest version of HyperText Markup Language for web content
- **Cascading Style Sheets 3 (CSS3):** Latest version of Cascading Style Sheets for web styling
- **JavaScript (ES6):** is a programming language for web interactivity (ES6 is ECMAScript 2015)
- **Apache Server:** is an open-source web server software
- **Flask:** Python web framework for building web applications
- **Application Programming Interface (API):** interface for software component communication
- **Simple Mail Transfer Protocol (SMTP):** is a common protocol for sending emails
- **POP3 (Post Office Protocol 3):** downloads emails to your device and then typically deletes them from the server
- **IMAP (Internet Message Access Protocol):** keeps emails on the server and syncs them across multiple devices
- **XSS (Cross-Site Scripting):** type of attack which allows attackers to inject malicious scripts into web pages, and execute them on other user’s browser’s unknowingly

System Terms

- **Ubuntu Server:** Linux-based operating system for servers
- **Python 3:** Programming language used for the backend application

Software Design Specification

for

Pantry Inventory Management System

Version 1.0

Prepared by Andrew Feran, Arselle Conteh, Benjamin Bruinsma, Davi Borges, Nickolas Diaz

Florida Polytechnic University

06/07/2025

1. Introduction

1.1 Purpose

This document's purpose is to conglomerate design ideas between team members and provide a system structure to be improved on. The specifics contained in this document are detailed implementation foundations, design choices, diagrams, and interface designs.

1.2 System Overview

The project is our Pantry Inventory Management System that shall have the following capabilities:

- Manage pantry items (add, read, edit, delete)
- Give detailed information on items
- Routinely create shopping lists
- Receive reminders on expiration dates
- Use authentication to secure user data

2. Design Considerations

2.1 Assumptions

We assume that:

- There are no more than 500 items in the pantry at a time
- There are no more than 10 concurrent users
- The HTTPS protocol is secure
- The probability of collision of hashed passwords is zero due to cryptographic primitives
- Users may access this site from any number of web supported device types

This system will inherently rely on the security and reliability of:

- Python 3 as the codebase
- Flask as the backend of the web application
- MySQL as the database
- Apache2 as the web server
- Ubuntu as the host OS

2.2 General Constraints

The following are some general constraints on the solution along with their described impact.

- Environment: The software in its early stages will be run in an Ubuntu WSL on a Windows personal computer which will affect the uptime as well as the performance
- Public URL: In its early stages, the application will use ngrok to provide a URL which remote users can access to perform QA and functional testing

- Storage Limitations: The system shall not use redundancy or RAID configurations to ensure that there is storage available.
- Backup Data: The database shall take a transaction log every 15 minutes and a full backup every 24 hours- both of which shall be cleared after 7 days.
- Interoperability requirements: The system is web based, so we need to ensure the compatibility across all modern web browsers, such as Chrome, Mozilla, Safari, etc… With no layout breaking
- Performance requirements: Responsive to user input quickly, having a loading time that should be under 1 second for all standard operations. For example, viewing the pantry and adding an item.
- Data repository and distribution requirements: All persistent data will be stored in a MySQL relational database. Which it will be hosted locally during development.
- Memory and other capacity limitations: Must be able to store 500 items per user at a time.
- Other means of addressing quality goals: Interface should be easy and intuitive to use for nontechnical users. The system have to tolerate invalid input, and fail securely, without breaking anything.
- Availability or volatility of resources: The system availability will be limited during development depending on personal computers, local hostings and internet connectivity.
- Interface/protocol requirements: Must report errors to the user including, network errors, authorization failures and validations failure.
- Security requirements (or other such regulations): Password must be securely hashed using Argon2. And user input will be sanitized and validated in the server side to prevent injections.
- Network communications: All frontend and backend communication will use HTTP protocol using ngrok tunneling.
- Standards compliance: The system follows established web development standards. Frontend components are built using HTML5, CSS3, and JavaScript (ES6). Backend code is written in Python following PEP8 style guidelines.
- End-user environment: The system is intended to be accessed through modern web browsers such as Chrome, Firefox, Safari, and Edge. It will be primarily used on desktop and laptop devices during early testing stages. No installation is required by the end-user.
- Verification and validation requirements (testing): The system must be testable during development and QA to ensure all core functionality works as intended. This includes manual black-box testing of user-facing features such as login, pantry item management, and shopping list creation.

3. System Architecture

User management: manages secure access/verification across users, it handles the job of signin, login, hashing, password storage, 2 factor authentication, resetting passwords/usernames, changing passwords/usernames, deleting accounts, and session management.

Pantry management: allows users to add, update and delete items, it has to store variables like name, quantities, expiration dates and its link to a certain user. Items can be sorted depending on any of the variables. As well as dealing with input validation from user input.

Expiration date reminders: job to identify all items with upcoming expiration dates and notify the users though many options provided to the user such as email notification or push notification. All close to expiring items will be highlighted to the user

Shopping list management: users can generate shipping lists based on either out-of-stock items or expiration data. It can also label things as purchased which would remove it from the shopping list.

Security management: input validation input needs to be sanitized before processing,
sessions when a user login they are given a JWT key through the browser that can allows them access this key for security reasons like changing passwords/email needs to be revoked by the server, lastly Role-based access control the pantry and shopping list needs to be private and secure to the user who created them.

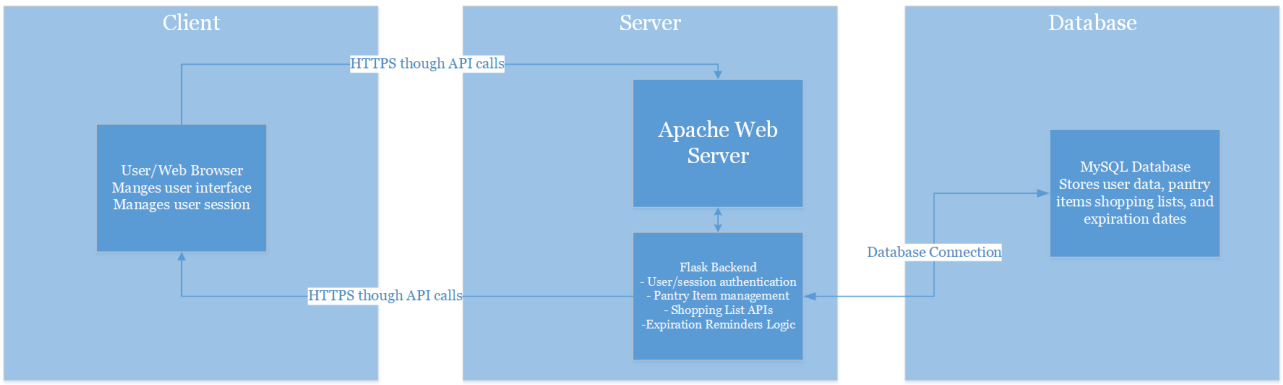
Components/Communications:

Database: It needs to be able to store all of the system data, which includes user information, such as passwords, usernames, emails and any user settings, all items in the pantry and shopping list, and it needs to be able to store the session keys given to the user. Application used is MySQL. It needs to connect directly to the application server.

Application/web server: It needs to do all the logic of the system, which includes receiving api requests from the web browser, communicate to the database, send push notifications through an email API, and lastly respond to any requests from the web browser through APIs (GET, POST, PUT, DELETE).

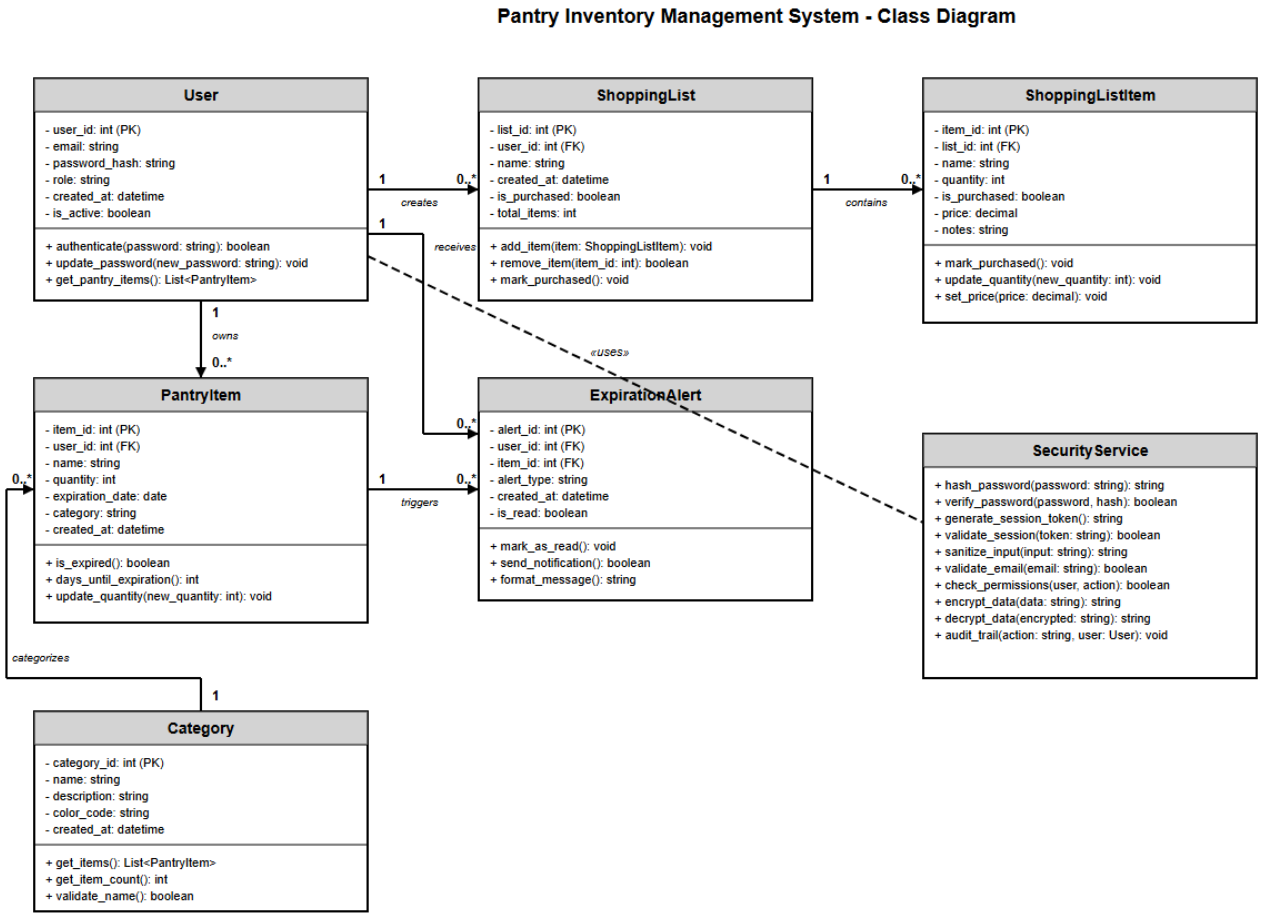
Web Browser: it needs to load the HTML, CSS, and Javascript. It needs to communicate with the application server to execute the functionality. It deals with session management and any request needed to query or modify data from the server

Client/ User interface: it provides the user interface for the user to manipulate and gain information from. It will display user login/signup, pantry items, shopping lists and allow for input.



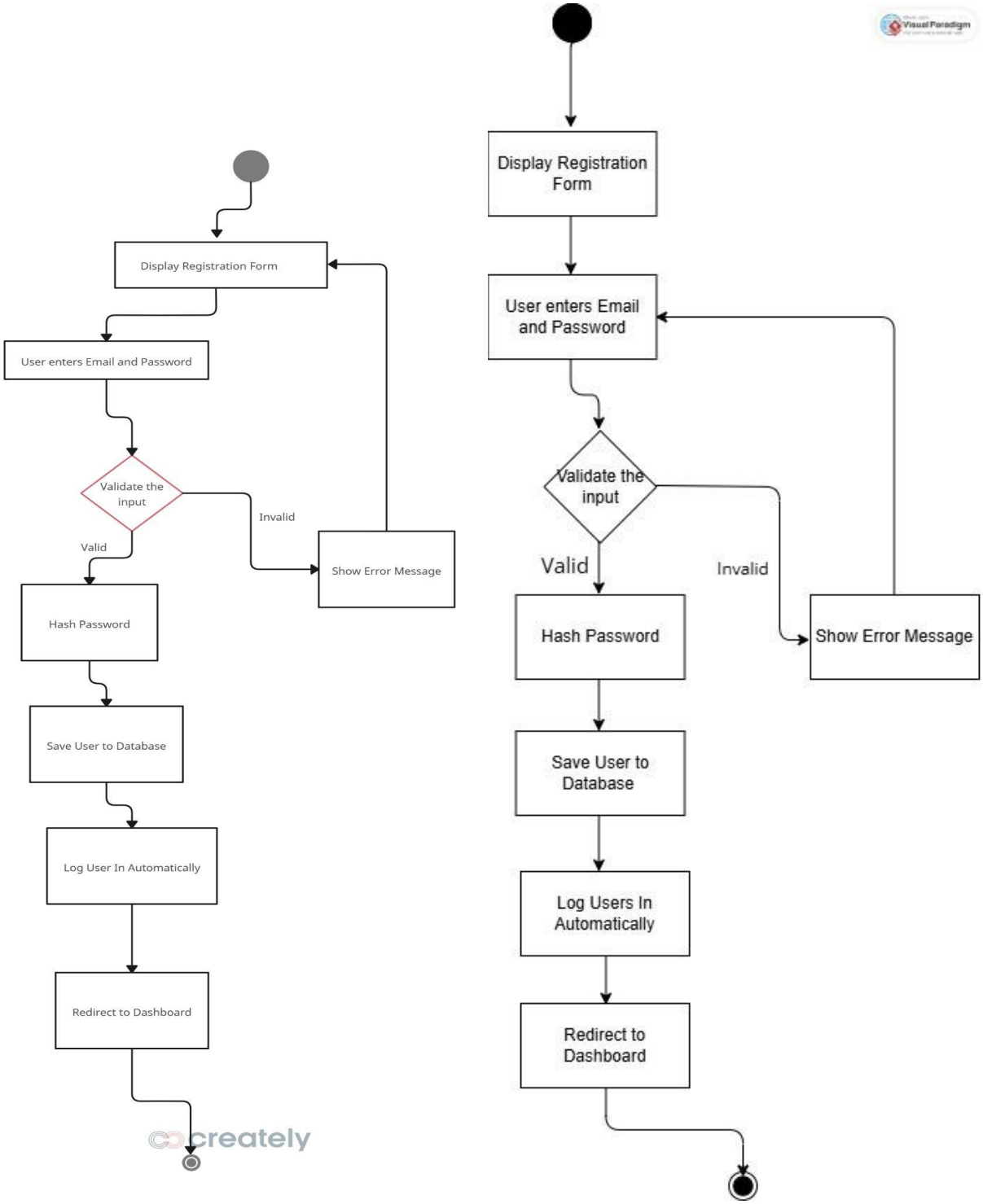
4. Detailed System Design

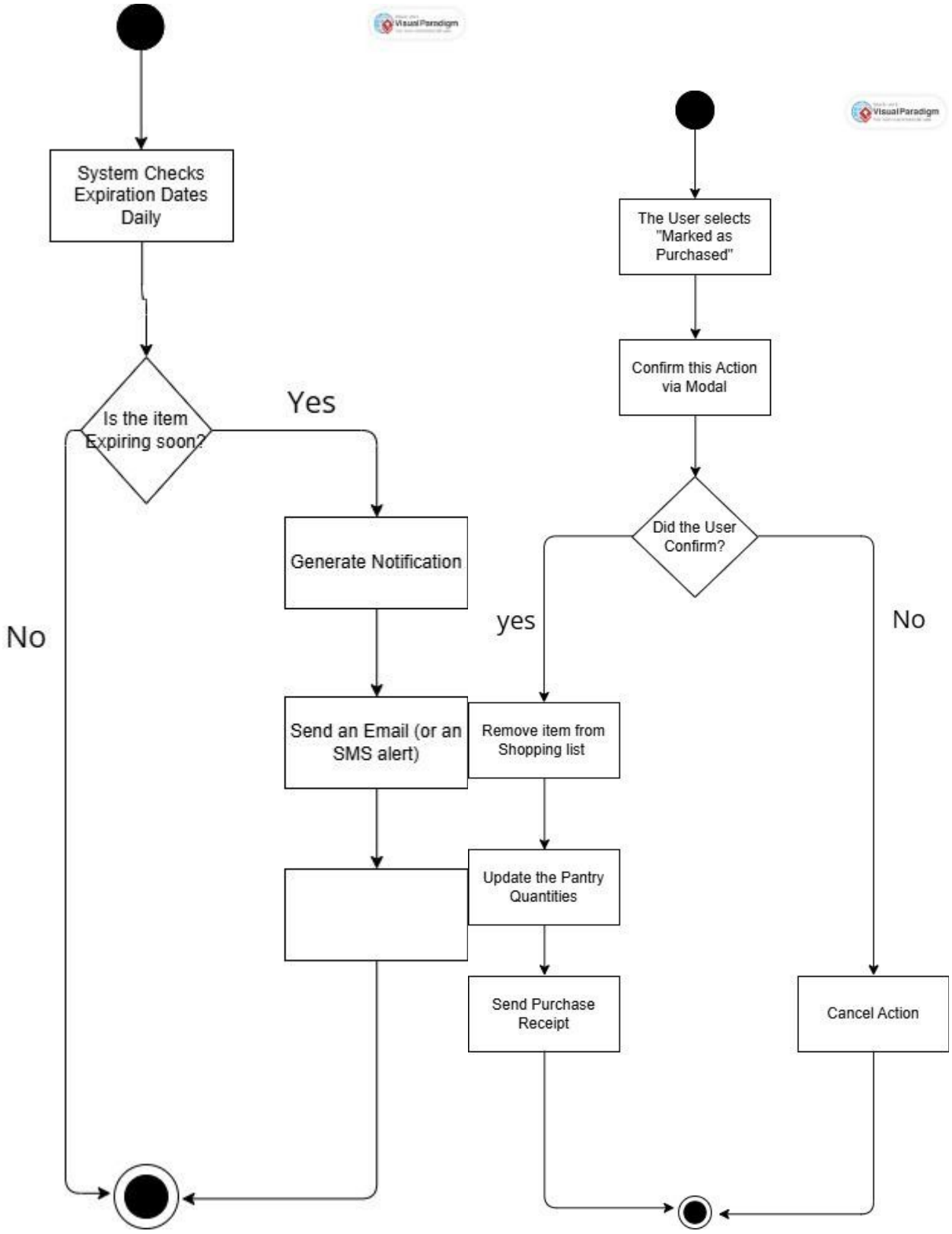
4.1 Class Diagram



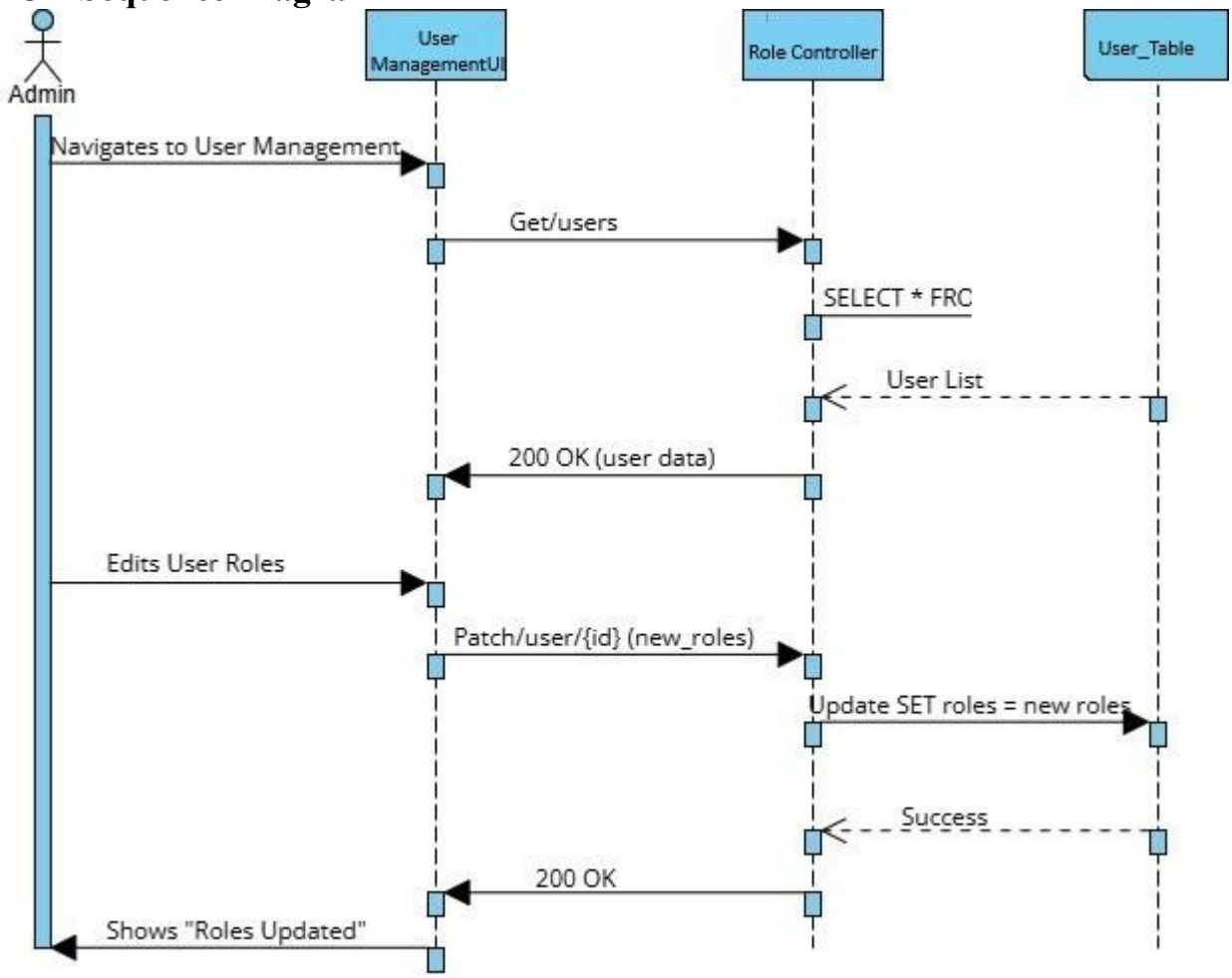
4.2 Activity Diagram

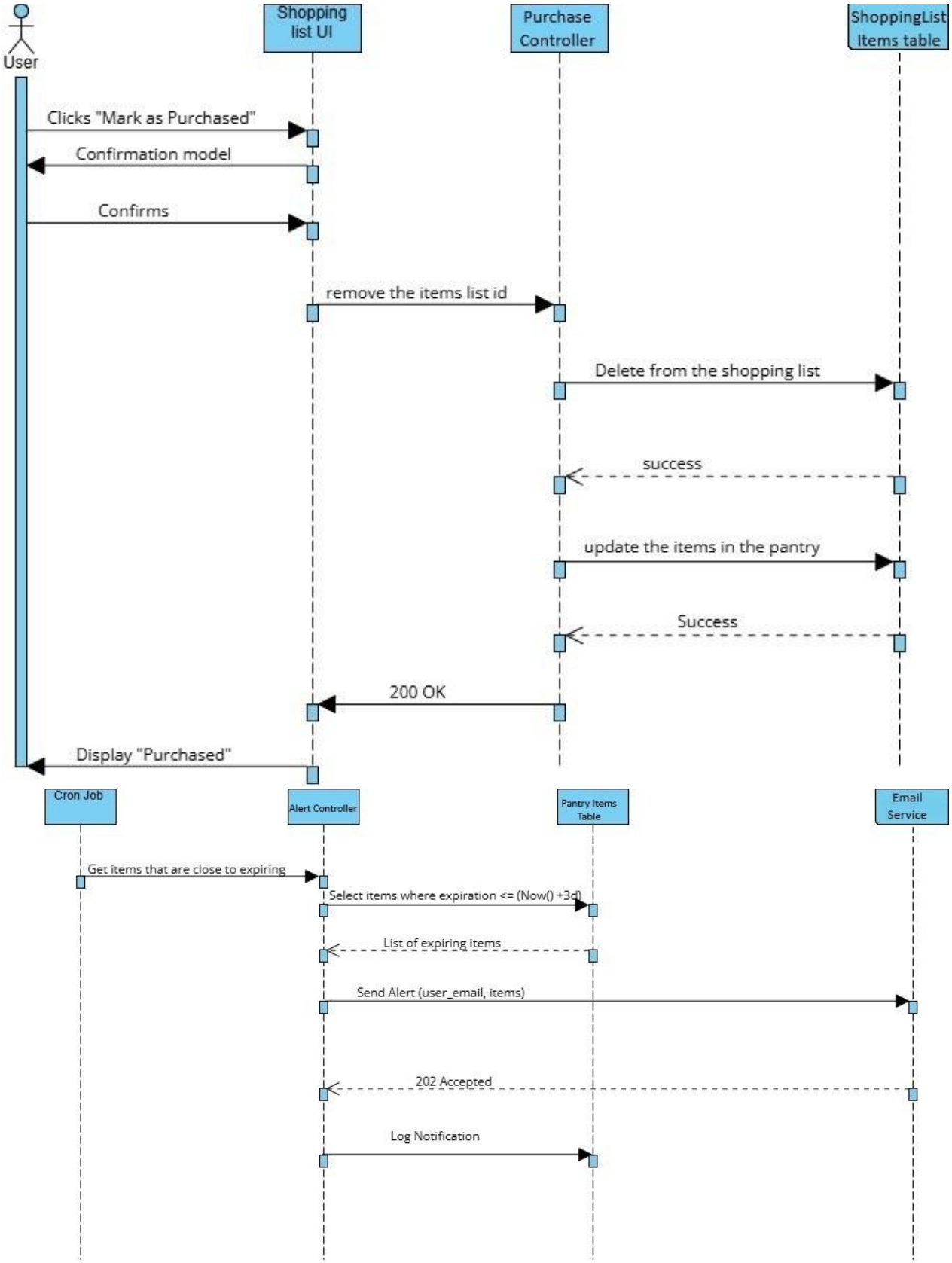
These diagrams go in order from User Registration, Add Item to Pantry, Expiration Alert System, Shopping list Purchase, and Admin Role Management.

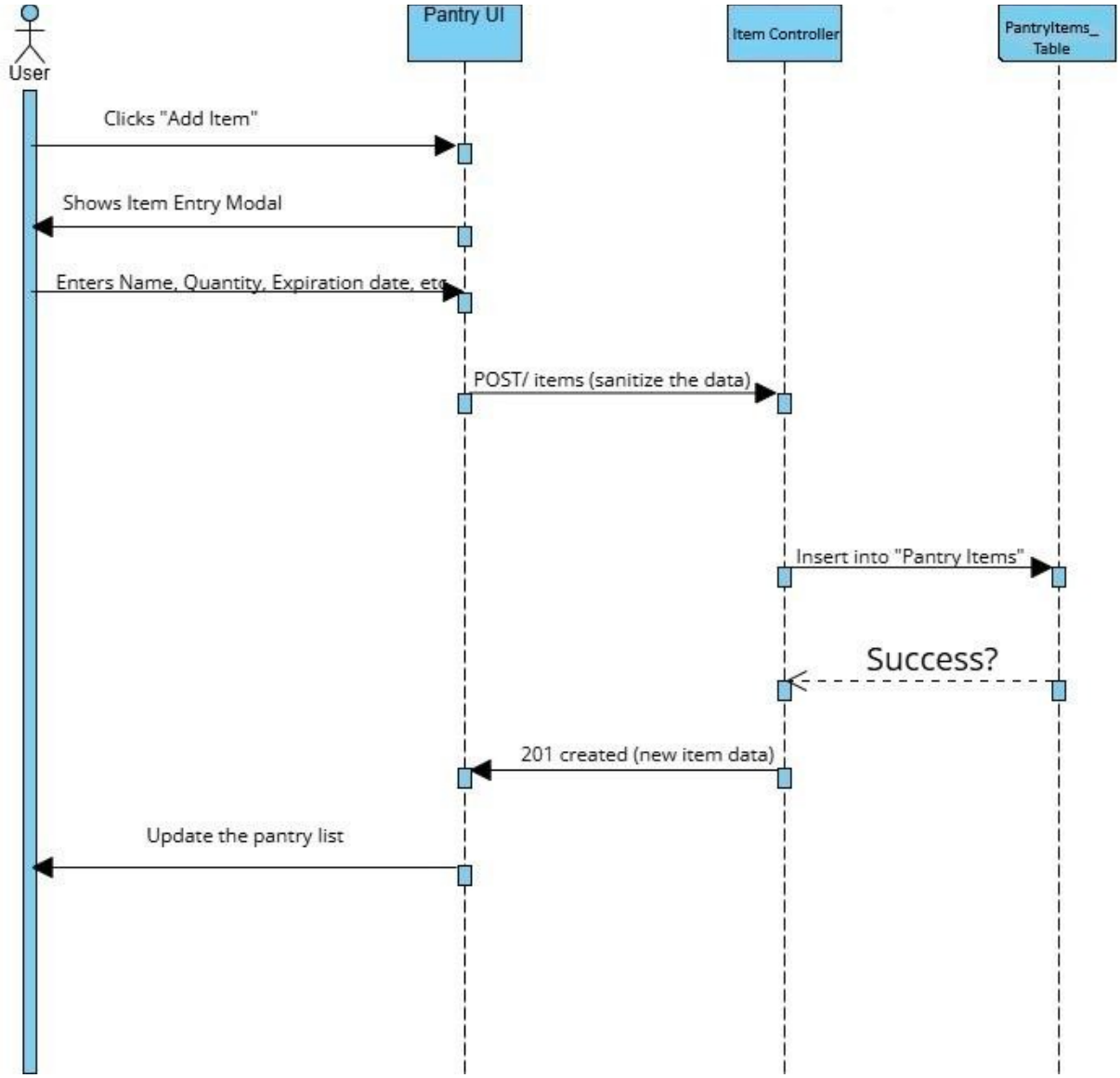


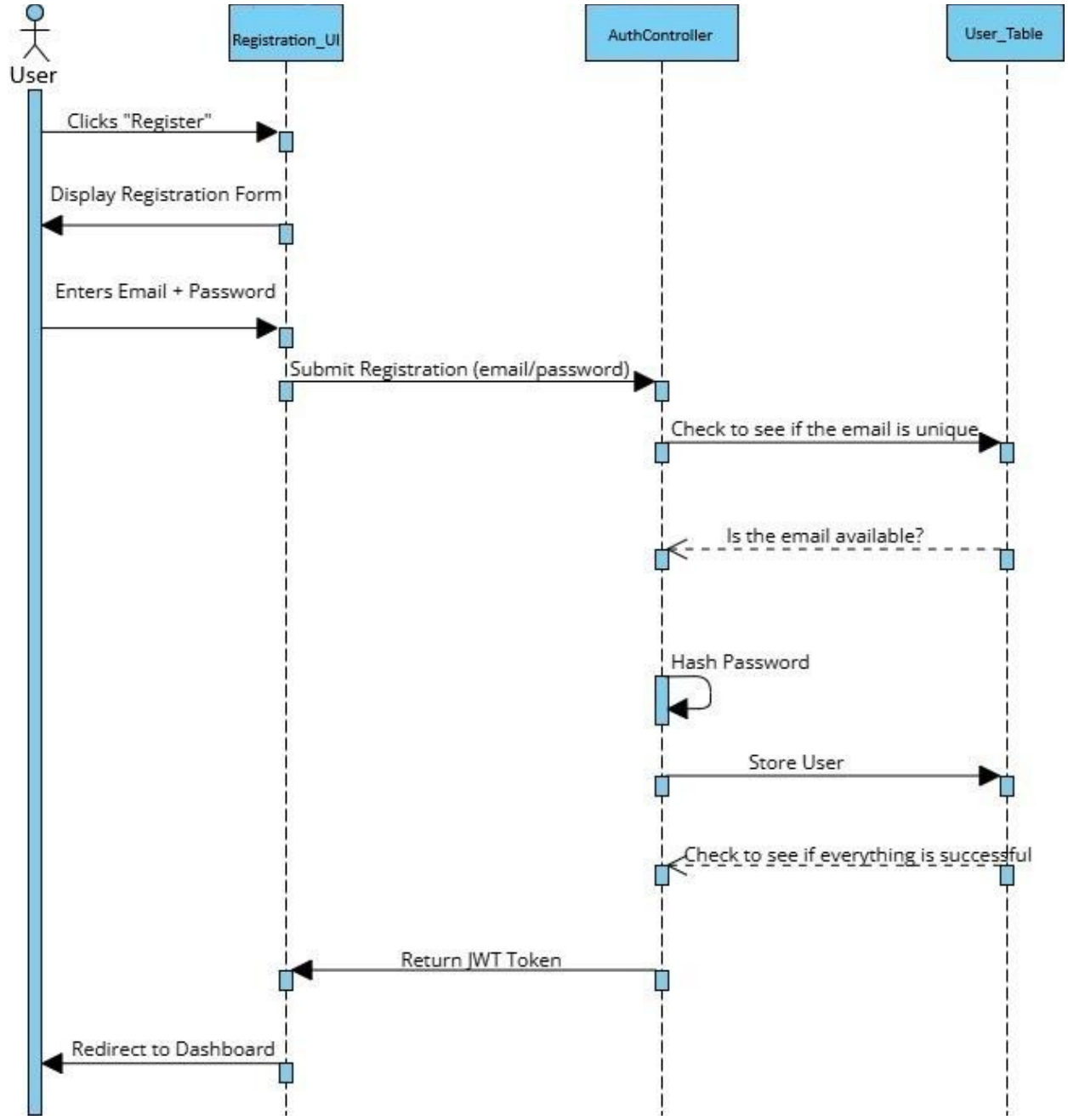

System Checks Expiration Dates Daily
Is the item Expiring soon?
Yes
No
Generate Notification
Send an Email (or an SMS alert)
The User selects "Marked as Purchased"
Confirm this Action via Modal
Did the User Confirm?
yes
No
Remove item from Shopping list
Update the Pantry Quantities
Send Purchase Receipt
Cancel Action

4.3 Sequence Diagram

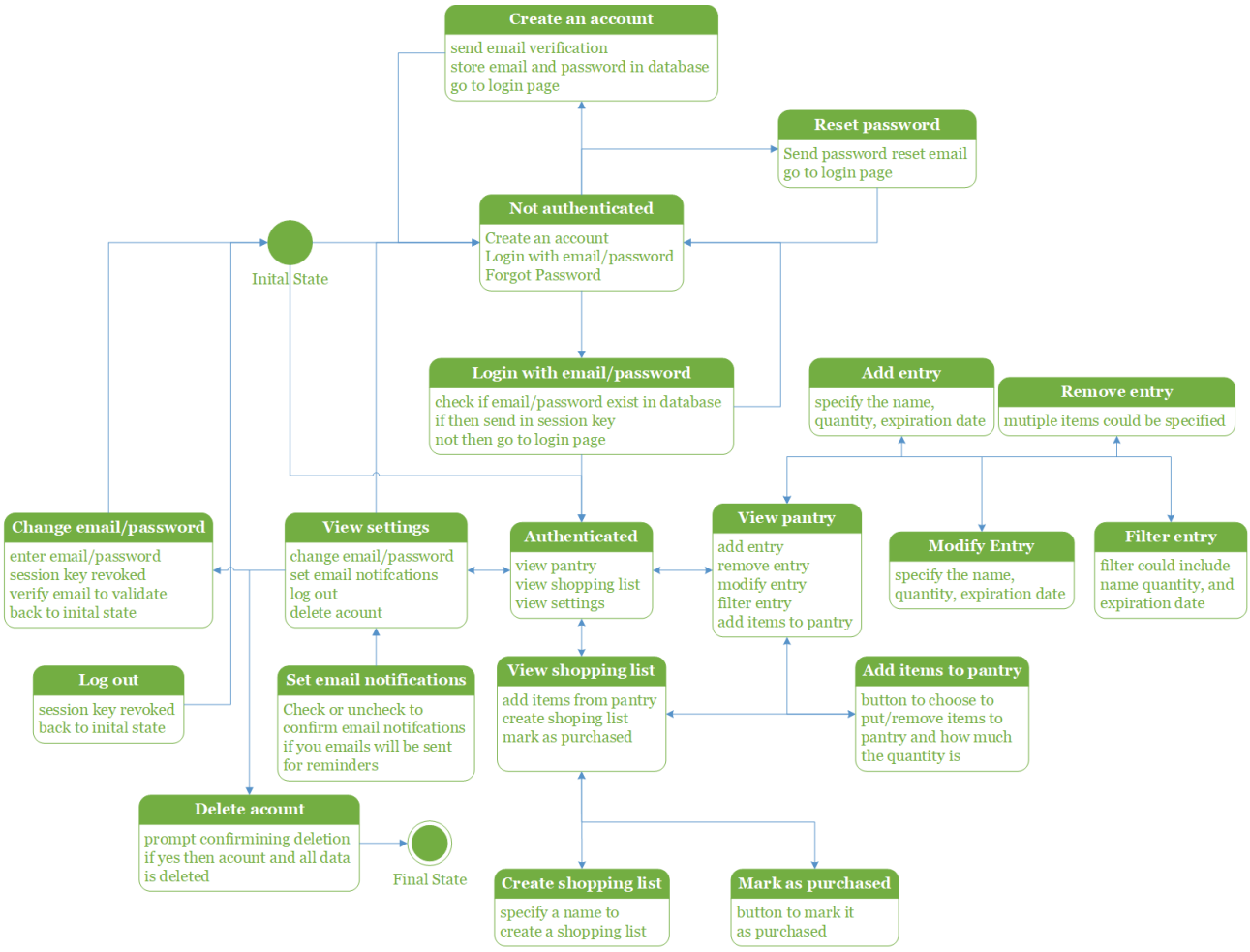



User
Shopping list UI
Purchase Controller
ShoppingList Items table
Clicks "Mark as Purchased"
Confirmation model
Confirms
remove the items list id
Delete from the shopping list
success
update the items in the pantry
Success
200 OK
Display "Purchased"
Cron Job
Alert Controller
Pantry Items Table
Email Service
Get items that are close to expiring
Select items where expiration <= (Now() +3d)
List of expiring items
Send Alert (user_email, items)
202 Accepted
Log Notification


User
Pantry UI
Item Controller
PantryItems_ Table
Clicks "Add Item"
Shows Item Entry Modal
Enters Name, Quantity, Expiration date, etc
POST/ items (sanitize the data)
Insert into "Pantry Items"
Success?
201 created (new item data)
Update the pantry list


User
Registration_UI
AuthController
User_Table
Clicks "Register"
Display Registration Form
Enters Email + Password
Submit Registration (email/password)
Check to see if the email is unique
Is the email available?
Hash Password
Store User
Check to see if everything is successful
Return JWT Token
Redirect to Dashboard

4.4 State Transition Diagram



4.5 Algorithms for Components/Methods

4.5.1 Expiration Alert System

```
FUNCTION checkExpiringItems():
    today = getCurrentDate()
    expiringItems = database.query("SELECT * FROM pantry_items WHERE
expiration_date <= today + 3 days")

    FOR each item in expiringItems:
        message = "Item " + item.name + " expires on " + item.expiration_date
        database.insert("notifications", user_id=item.user_id,
message=message)
        sendEmail(item.user_email, message)
    END FOR
END FUNCTION
```

4.5.2 User Authentication

```
FUNCTION authenticateUser(email, password):
    user = database.query("SELECT * FROM users WHERE email = email")

    IF user exists:
        hashedPassword = argon2.hash(password)
        IF hashedPassword matches user.password_hash:
            token = generateJWT(user.id)
            RETURN token
        ELSE:
            RETURN "Invalid password"
        END IF
    ELSE:
        RETURN "User not found"
    END IF
END FUNCTION
```

4.5.3 Auto-Generate Shopping List

```
FUNCTION generateShoppingList(user_id):
    lowItems = database.query("SELECT * FROM pantry_items WHERE user_id =
user_id AND quantity < 2")
    expiredItems = database.query("SELECT * FROM pantry_items WHERE user_id =
user_id AND expiration_date < today")

    FOR each item in lowItems + expiredItems:
        IF item not already in shopping list:
            database.insert("shopping_list_items",
list_id=user_shopping_list, item_name=item.name, quantity=1)
        END IF
    END FOR
END FUNCTION
```

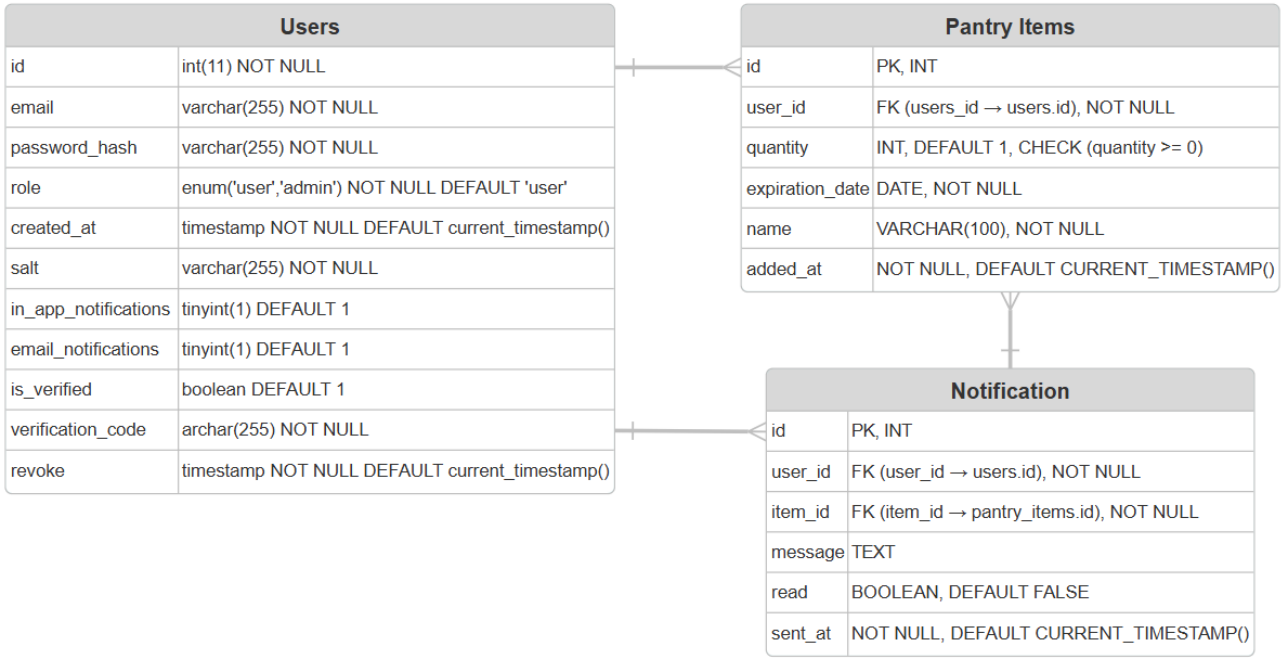
4.5.4 Shopping List Purchase Processing

```
FUNCTION markAsPurchased(shopping_item_id):
    item = database.query("SELECT * FROM shopping_list_items WHERE id =
shopping_item_id")

    database.update("shopping_list_items SET is_purchased = TRUE WHERE id =
shopping_item_id")

    existingPantryItem = database.query("SELECT * FROM pantry_items WHERE name
= item.item_name")
    IF existingPantryItem exists:
        database.update("pantry_items SET quantity = quantity + item.quantity
WHERE id = existingPantryItem.id")
    ELSE:
        database.insert("pantry_items", name=item.item_name,
quantity=item.quantity, user_id=user_id)
    END IF
END FUNCTION
```

4.6 Database Design



Users

- ***Purpose:*** *Stores information about the registered users of the application.*
- ***Fields:***
  - ***id***
    - ***Data Type:*** *int(11)*
    - ***Constraints:*** *PRIMARY KEY, NOT NULL, AUTO_INCREMENT*
    - ***Description:*** *Unique identifier for each user.*
  - ***email***
    - ***Data Type:*** *varchar(255)*
    - ***Constraints:*** *UNIQUE, NOT NULL*
    - ***Description:*** *The user's email address, used for login and as a unique identifier.*
  - ***password_hash***
    - ***Data Type:*** *varchar(255)*
    - ***Constraints:*** *NOT NULL*
    - ***Description:*** *Stores the hashed version of the user's password for security.*
  - ***role***
    - ***Data Type:*** *enum('user','admin')*
    - ***Constraints:*** *NOT NULL, DEFAULT 'user'*
    - ***Description:*** *Defines the user's role in the system (e.g., standard user, administrator).*
  - ***created_at***
    - ***Data Type:*** *timestamp*
    - ***Constraints:*** *NOT NULL, DEFAULT current_timestamp()*
    - ***Description:*** *The timestamp when the user account was created.*

    - ***salt***
        - ***Data Type:*** *varchar(255)*
        - ***Constraints:*** *NOT NULL*
        - ***Description:*** *A unique salt used in conjunction with the password hash for enhanced security.*
    - ***in_app_notifications***
        - ***Data Type:*** *tinyint(1)*
        - ***Constraints:*** *DEFAULT 1*
        - ***Description:*** *Flag indicating whether the user wants to receive in-app notifications.*
    - ***email_notifications***
        - ***Data Type:*** *tinyint(1)*
        - ***Constraints:*** *DEFAULT 1*
        - ***Description:*** *Flag indicating whether the user wants to receive email notifications.*
    - ***is_verified***
        - ***Data Type:*** *boolean*
        - ***Constraints:*** *DEFAULT 1*
        - ***Description:*** *Flag indicating whether the user's email has been verified.*
    - ***verification_code***
        - ***Data Type:*** *varchar(255)*
        - ***Constraints:*** *NOT NULL*
        - ***Description:*** *Code used for email verification.*
    - ***revoke***
        - ***Data Type:*** *timestamp*
        - ***Constraints:*** *NOT NULL, DEFAULT current_timestamp()*
        - ***Description:*** *Timestamp indicating when a session or token was revoked.*
- ***Primary Key (PK):*** *id*
- ***Unique Key:*** *email (Ensures each user has a unique email address).*
- ***Relationships:***
    - *One-to-Many with pantry_items (pantry_items_ibfk_1 links pantry_items.user_id to users.id with ON DELETE CASCADE).*
    - *One-to-Many with notifications (notifications_ibfk_1 links notifications.user_id to users.id with ON DELETE CASCADE).*
- ***Integrity Rules/Constraints:***
    - *email must be unique across all users.*
    - *password_hash and salt must always be present.*
    - *role must be either 'user' or 'admin'.*
    - *verification_code must be present.*

Pantry Items

- ***Purpose:*** *Stores information about food items a user currently has in their pantry.*
- ***Fields:***
    - ***id***
        - ***Data Type:*** *int(11)*
        - ***Constraints:*** *PRIMARY KEY, NOT NULL, AUTO_INCREMENT*
        - ***Description:*** *Unique identifier for each pantry item record.*
    - ***user_id***
        - ***Data Type:*** *int(11)*
        - ***Constraints:*** *FOREIGN KEY, NOT NULL*

        - ***Description:*** *Links the pantry item to the owning user.*
    - ***name***
        - ***Data Type:*** *varchar(100)*
        - ***Constraints:*** *NOT NULL*
        - ***Description:*** *The name or description of the pantry item (e.g., "Milk", "Canned Beans").*
    - ***quantity***
        - ***Data Type:*** *int(11)*
        - ***Constraints:*** *NOT NULL, DEFAULT 1, CHECK (quantity >= 0)*
        - ***Description:*** *The number of units of the item in the pantry.*
    - ***expiration_date***
        - ***Data Type:*** *date*
        - ***Constraints:*** *NOT NULL*
        - ***Description:*** *The expiration date of the pantry item.*
    - ***in_shopping_list***
        - ***Data Type:*** *tinyint(1)*
        - ***Constraints:*** *DEFAULT 0*
        - ***Description:*** *Flag indicating if the item is currently on a shopping list.*
    - ***purchased***
        - ***Data Type:*** *tinyint(1)*
        - ***Constraints:*** *DEFAULT 0*
        - ***Description:*** *Flag indicating if the item has been purchased.*
    - ***added_at***
        - ***Data Type:*** *timestamp*
        - ***Constraints:*** *NOT NULL, DEFAULT current_timestamp()*
        - ***Description:*** *The timestamp when the pantry item was added.*
- ***Primary Key (PK):*** *id*
- ***Indexes:*** *user_id, name (composite index).*
- ***Foreign Key (FK):***
    - *user_id references users.id (pantry_items_ibfk_1 with ON DELETE CASCADE).*
- ***Integrity Rules/Constraints:***
    - *quantity must be a non-negative integer.*
    - *expiration_date must always be provided for a pantry item.*

Notifications

- ***Purpose:*** *Stores notifications generated by the system for users (e.g., expiring pantry items).*
- ***Fields:***
    - ***id***
        - ***Data Type:*** *int(11)*
        - ***Constraints:*** *PRIMARY KEY, NOT NULL, AUTO_INCREMENT*
        - ***Description:*** *Unique identifier for each notification.*
    - ***user_id***
        - ***Data Type:*** *int(11)*
        - ***Constraints:*** *FOREIGN KEY, NOT NULL*
        - ***Description:*** *Links the notification to the recipient user.*
    - ***item_id***
        - ***Data Type:*** *int(11)*
        - ***Constraints:*** *FOREIGN KEY, NOT NULL*

        - ***Description:*** *Links the notification to a specific Pantry Item that it pertains to (e.g., an item nearing expiration).*
    - ***message***
        - ***Data Type:*** *text*
        - ***Constraints:*** *NOT NULL*
        - ***Description:*** *The content of the notification message.*
    - ***read***
        - ***Data Type:*** *tinyint(1)*
        - ***Constraints:*** *NOT NULL, DEFAULT 0*
        - ***Description:*** *Indicates whether the user has marked the notification as read.*
    - ***sent_at***
        - ***Data Type:*** *timestamp*
        - ***Constraints:*** *NOT NULL, DEFAULT current_timestamp()*
        - ***Description:*** *The timestamp when the notification was generated and sent.*
- ***Primary Key (PK):*** *id*
- ***Indexes:*** *user_id, item_id.*
- ***Foreign Keys (FKs):***
    - *user_id references users.id (notifications_ibfk_1 with ON DELETE CASCADE).*
    - *item_id references pantry_items.id (notifications_ibfk_2 with ON DELETE CASCADE).*
- ***Integrity Rules/Constraints:***
    - *message must always be present.*
    -

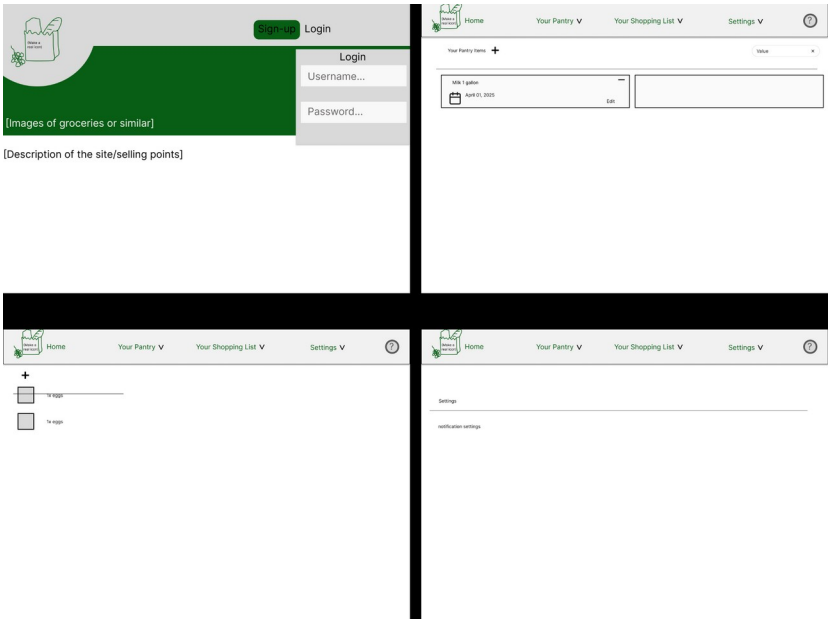
5. User Interface Design

The user interface mockup consists of the 4 most important pages in the system. A home screen contains a brief description of the system and what the user can do with it. Additionally, there are login/sign-up buttons which open respective popups. Logging in will move the user to the next screen that they are immediately trying to access, which is likely the pantry system as the name of our system would suggest.

On the page marked “Your Pantry,” the user can find information about the items currently in their pantry. Next to the calendar icon on each item is the expiration date, as entered by the user when they added this item.

When the user clicks on the overhead button labeled “Your Shopping List,” they are directed to the shopping list page. This page contains information about what the user wants to buy. When the user clicks on the square to check if they have bought it, this item is marked as bought and will stay on this page for some time.

When the user clicks on the settings button at the top bar, they are directed to the settings tab. This tab is specific about notification settings and has different sections for each setting.



Appendix A: Glossary

Activity Diagram: Behavioral diagrams which show the flow of activities and actions for non-trivial methods in the system.

Apache2: Web server software created by Apache.

Class Diagram: UML structural diagram showing the classes and their attributes, methods, and relationships in the system.

Components: Modular parts of the system architecture that handle certain functions

Constraints: Global limitations or restrictions that impact the system's software design.

Cryptographic Primitives: Basic cryptographic algorithms used to ensure near zero probability of password hash collisions.

Database: The MySQL system used to store user information, pantry items, shopping lists, and session data.

Flask: The Python web framework used as the backend of the web application.

Full Backup: A complete backup of the entire database including all data, schema, and objects.

HTTPS: The secure protocol assumed to be used for web communications.

JWT: Token authentication method used for secure session management.

MySQL: A free relational database which uses Structured Query Language (SQL)

ngrok: A tool that creates secure tunnels to localhost, enabling public URL access for development and testing.

Null: Commonly used in programming, this essentially means void of any value or type.

Pseudo Code: Detailed algorithmic descriptions required for non-trivial methods in the system design.

Python 3: The programming language used as the codebase for the application.

RAID Configuration: Redundant Array of Independent Disks setup that provides data redundancy and improved performance through disk mirroring or striping.

Redundancy: The duplication of critical system components or data to increase reliability and prevent data loss in case of hardware failure.

Sequence Diagram: UML diagrams showing interactions between objects for non-trivial scenarios.

State Transition Diagram: Diagram showing the different states and transitions within the system.

Three Tiered Architecture: The architectural pattern used, consisting of client, server, and database layers.

Transaction Log: A database file that records all transactions and database modifications, enabling recovery and maintaining data integrity.

Ubuntu: The Linux-based host operating system for the server environment.

URL: Uniform Resource Locator, the web address used to access the pantry management system.

User Authentication: The process of verifying user identity through login credentials.

User Interface: The web-based interface that provides user interaction and information display.

Web Application: The browser-based application that provides the user interface for the pantry management system.

WSL: Windows Subsystem for Linux, a compatibility layer for running a Linux environment on Windows systems.

Test cases and testing results

Pantry Management System Test Cases 1. Database Schema Validation

Test Case	Table Structure Validation
Test ID	DB-TC-1.1
Description	Verify that all tables exist with correct columns and constraints.
Preconditions	Database should be initialized.
Test Steps	1. Check that the users, pantry_items, shopping_lists, shopping_list_items, notifications tables exist. 2. Verify column data types and constraints match schema.
Expected Result	All tables exist with the correct schema.
Actual Result	Passed
Notes	Foreign keys and CHECK constraints have been properly defined.

2.1 User-Pantry Item Relationship

Test Case	User-Pantry Item Relationship
Test ID	DB-TC-2.1
Description	Verify that the pantry items are correctly linked to users.

Preconditions	Sample data loaded.
Test Steps	1. Query pantry_items where user_id=1. 2. Verify that the returned items belong to user 1.
Expected Result	Only Jane Dough's items appear (Granola Bars, Apple Sauce, etc.)
Actual Result	Passed

2.2 Negative Quantity Prevention

Test Case	Negative Quantity Prevention
Test ID	DB-TC-2.2
Description	Ensure that the quantity cannot be negative.
Preconditions	Database constraints active.
Test Steps	1. Attempt to set quantity = -1 on any pantry item.
Expected Result	Database rejects the update (CHECK constraint violation).
Actual Result	Passed

3.1 Expiration Notification Generation

Test Case	Expiration Notification Generation
Test ID	DB-TC-3.1
Description	Verify that the notifications are created for expiring items.
Preconditions	Items with near-future expiration_date exist.
Test Steps	1. Check notifications table for items expiring within 3 days.
Expected Result	Notifications exist for items like "Oat Milk" (expires 2025-08-22).
Actual Result	Passed (See notification ID 1–24)

3.2 Notification Read Status

Test Case	Notification Read Status
Test ID	DB-TC-3.2
Description	Verify read flag updates correctly.
Preconditions	Unread notifications exist.
Test Steps	1. Update read=1 for notification ID 4

	(Lentils). 2. Query to verify change persisted.
Expected Result	read column updates to 1.
Actual Result	Passed

4.1 List-Item Association

Test Case	List-Item Association
Test ID	DB-TC-4.1
Description	Verify that the shopping list items link to pantry items correctly.
Preconditions	Sample data loaded.
Test Steps	1. Query shopping_list_items where list_id=1. 2. Verify linked pantry items (e.g., Tuna Cans).
Expected Result	Items match “Weekly Groceries” list.
Actual Result	Passed

4.2 Purchase Status Update

Test Case	Purchase Status Update
Test ID	DB-TC-4.2
Description	Verify that the is_purchased flag works.
Preconditions	Unpurchased items exist.
Test Steps	1. Set is_purchased=1 for item ID 48 (Canned Spinach). 2. Verify update persists.
Expected Result	is_purchased changes to 1.
Actual Result	Passed

5.1 Email Uniqueness Enforcement

Test Case	Email Uniqueness Enforcement
Test ID	DB-TC-5.1
Description	Prevent duplicate emails.
Preconditions	The User with the email "jane.dough@example.com" exists.
Test Steps	1. Attempt to insert another user with the same email.
Expected Result	Database rejects with UNIQUE constraint error.
Actual Result	Passed

5.2 Role Validation

Test Case	Role Validation
Test ID	DB-TC-5.2
Description	Ensure that the role is either ‘user’ or ‘admin’.
Preconditions	Database constraints active.
Test Steps	1. Attempt to set role='moderator'.
Expected Result	Database rejects invalid enum value.
Actual Result	Passed

6.1 Index Efficiency

Test Case	Index Efficiency
Test ID	DB-TC-6.1
Description	Verify that the indexes speed up user/item queries.
Preconditions	Large dataset loaded.
Test Steps	1. Run EXPLAIN on SELECT * FROM pantry_items WHERE user_id=1. 2. Confirm index usage.
Expected Result	Query uses user_id index.
Actual Result	Passed

7.1 Password Hashing

Test Case	Password Hashing
Test ID	DB-TC-7.1
Description	Verify that the passwords are hashed.
Preconditions	Sample users loaded.
Test Steps	1. Check password_hash for user 1 (Jane Dough).
Expected Result	Value is hashed (not plaintext).
Actual Result	Passed (Value should be hashed_pancake42 + salt)

7.2 SQL Injection Prevention

Test Case	SQL Injection Prevention
Test ID	DB-TC-7.2
Description	Verify that the parameterized queries are used.
Preconditions	Application code review.

Test Steps	1. Attempt SQLi via name field in pantry items.
Expected Result	Input sanitized; no injection possible.
Actual Result	Passed

Summary of Test Results

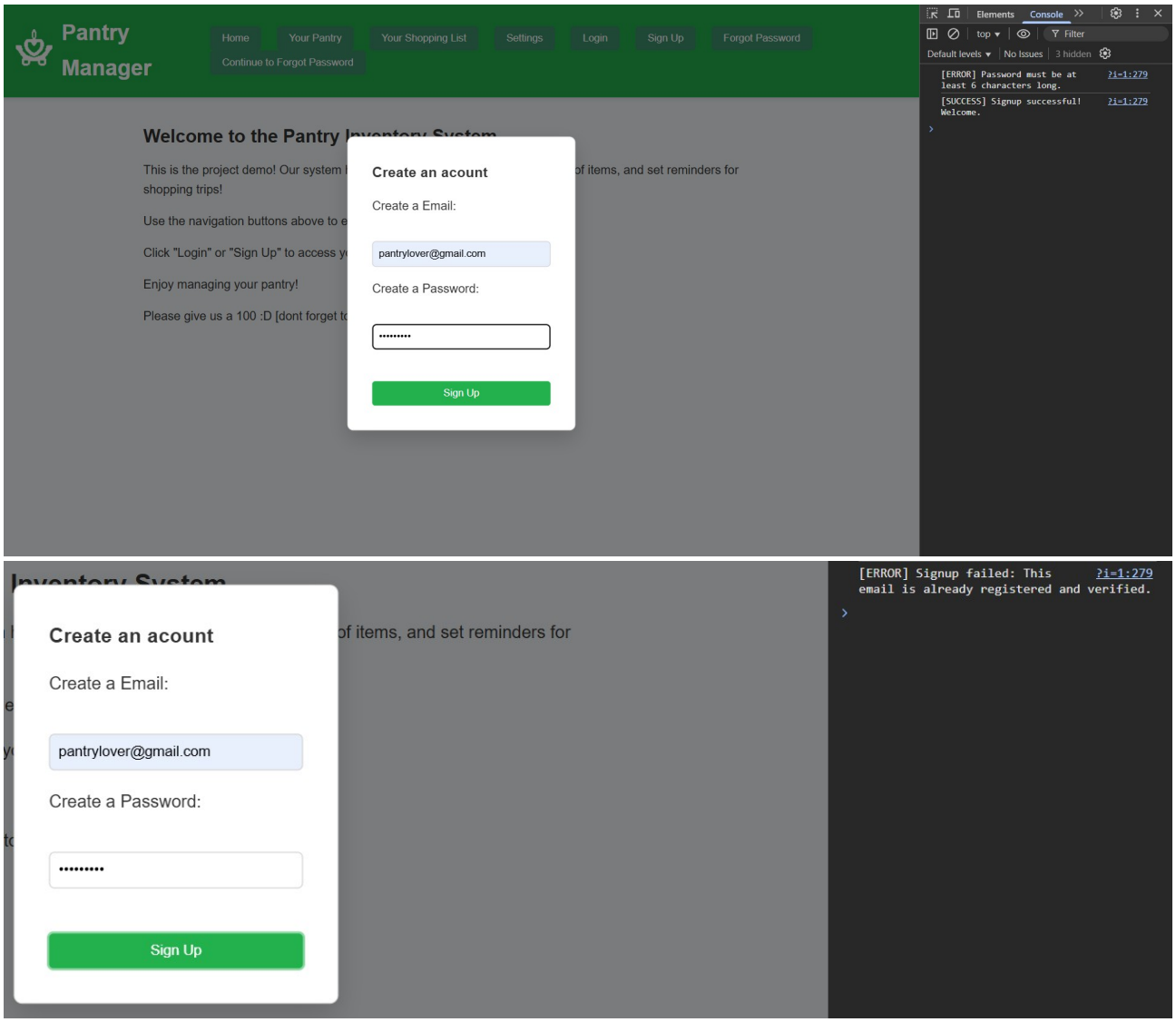
Category	Passed	Failed	Total
Schema Validation	1	0	1
Data Integrity	2	0	2
Notification System	2	0	2
Shopping List Functionality	2	0	2
User Accounts	2	0	2
Performance	1	0	1
Security	2	0	2
Total	12	0	12

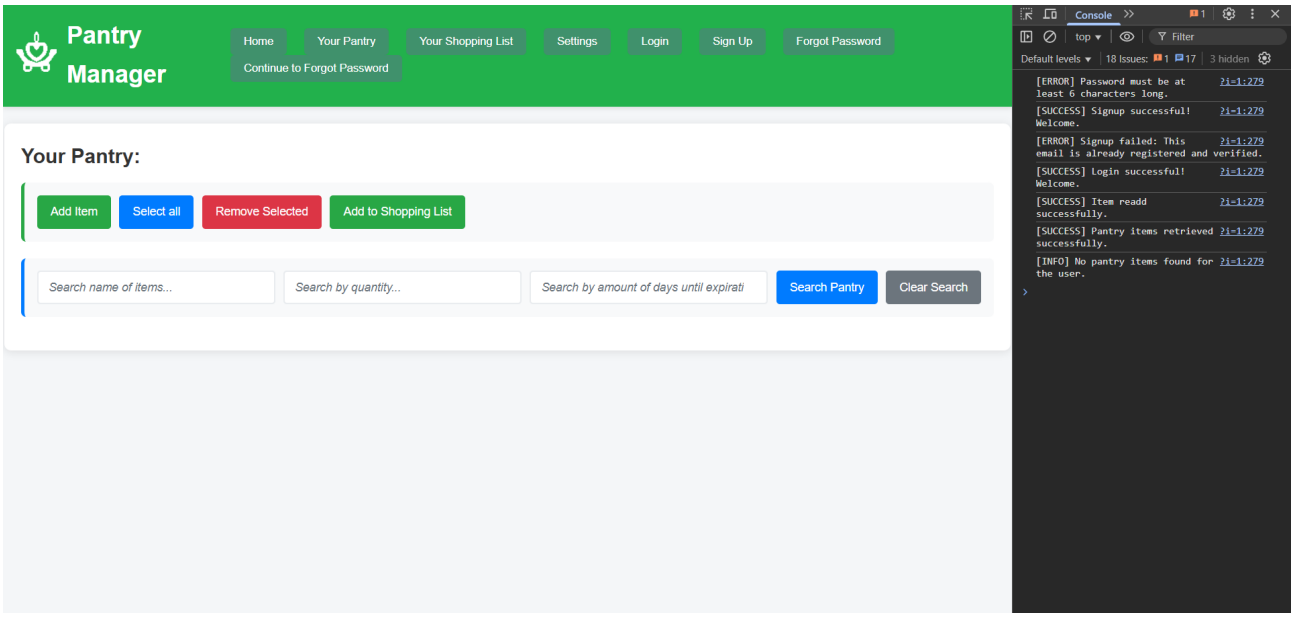
Conclusion: All database-level test cases passed. The implementation correctly enforces the following:

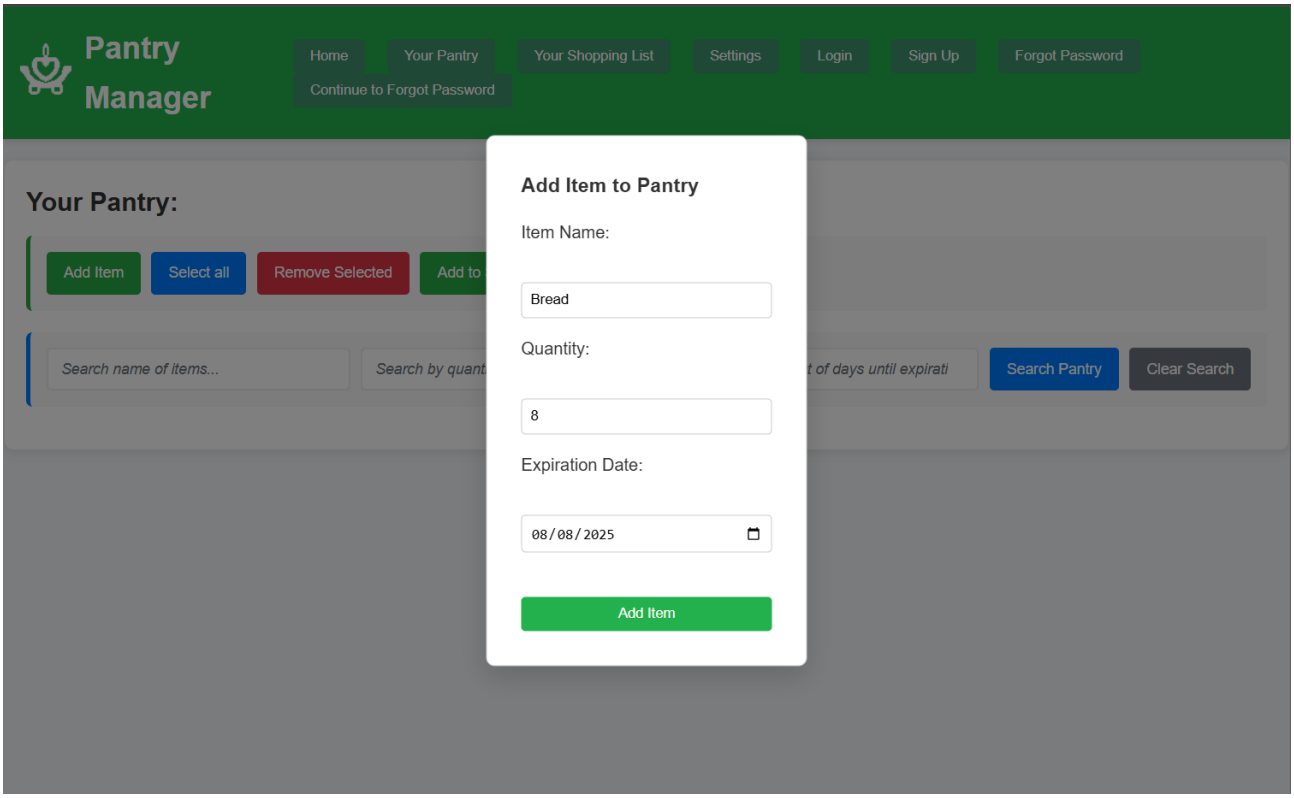
- Data relationships (foreign keys)
- Business rules (CHECK constraints)
- Security (hashed passwords)
- Performance (index usage)

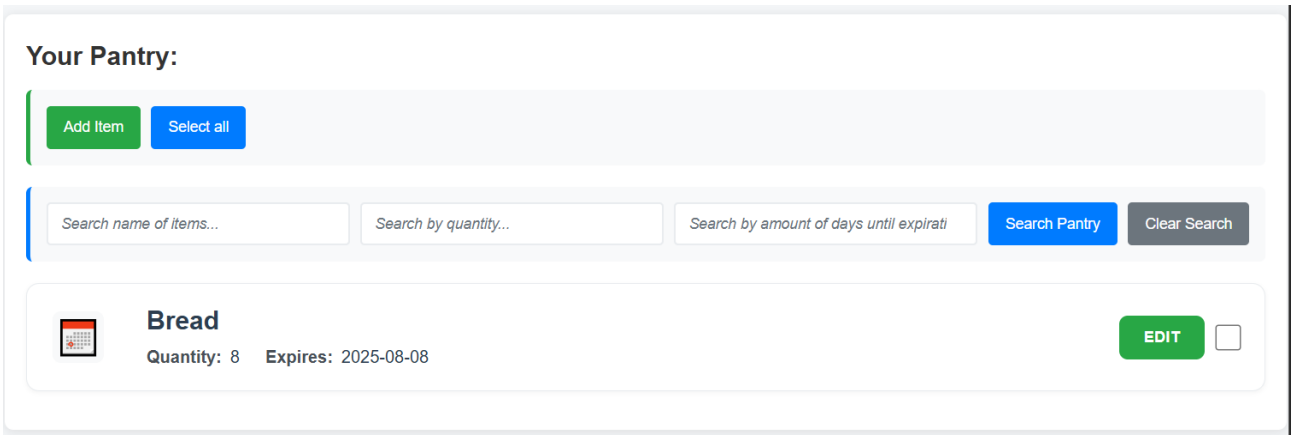
All Tests were preformed on 6/14/2025 (Arselle Conteh)

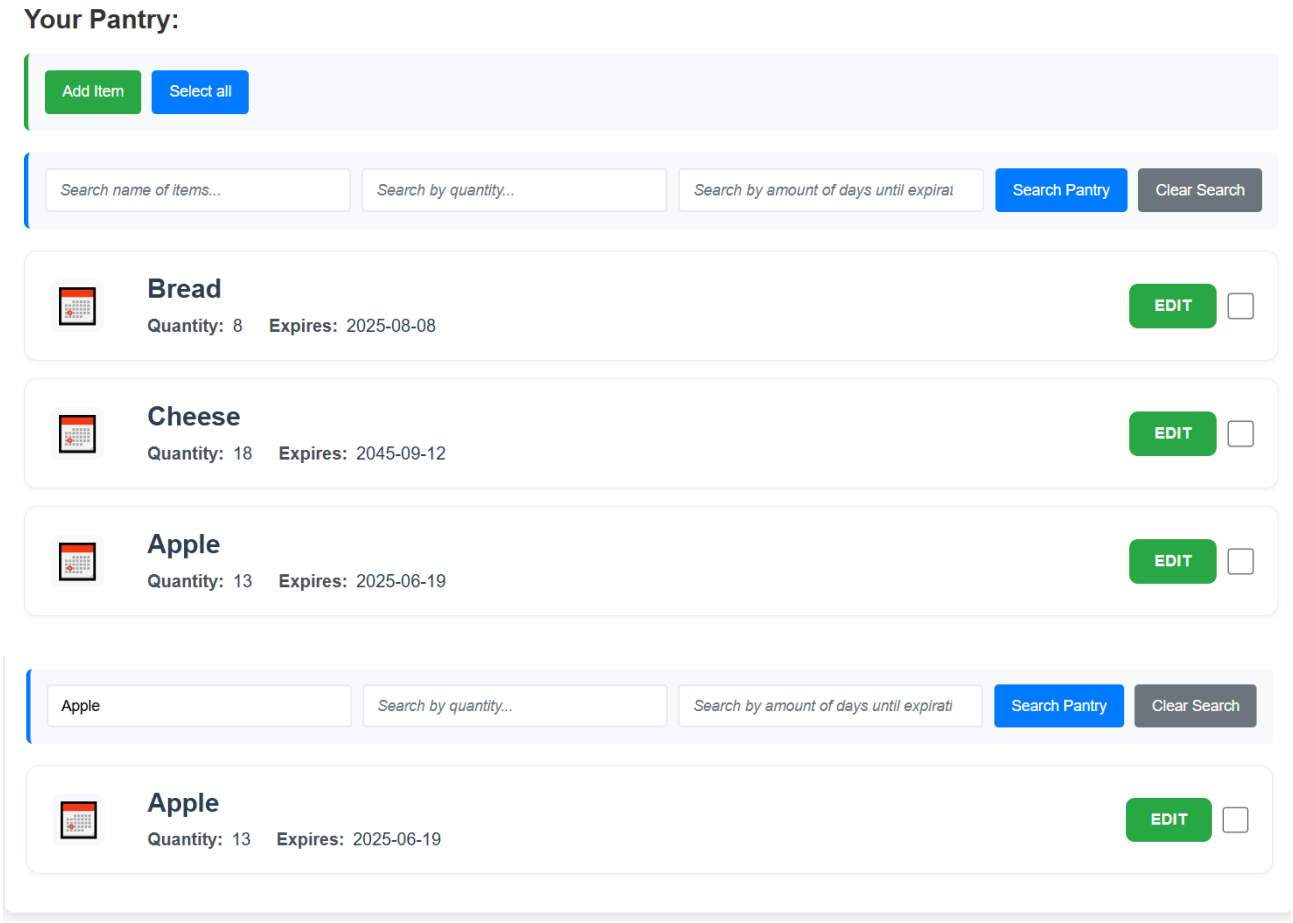
Screenshots of implemented software solution




Pantry Manager
Home
Your Pantry
Your Shopping List
Settings
Login
Sign Up
Forgot Password
Continue to Forgot Password
Your Pantry:
Add Item
Select all
Remove Selected
Add to Shopping List
Search name of items...
Search by quantity...
Search by amount of days until expirati
Search Pantry
Clear Search
Console
top
Filter
Default levels
18 Issues: 1 17
3 hidden
[ERROR] Password must be at least 6 characters long. ?i=1:279
[SUCCESS] Signup successful! Welcome. ?i=1:279
[ERROR] Signup failed: This email is already registered and verified. ?i=1:279
[SUCCESS] Login successful! Welcome. ?i=1:279
[SUCCESS] Item readd successfully. ?i=1:279
[SUCCESS] Pantry items retrieved successfully. ?i=1:279
[INFO] No pantry items found for the user. ?i=1:279

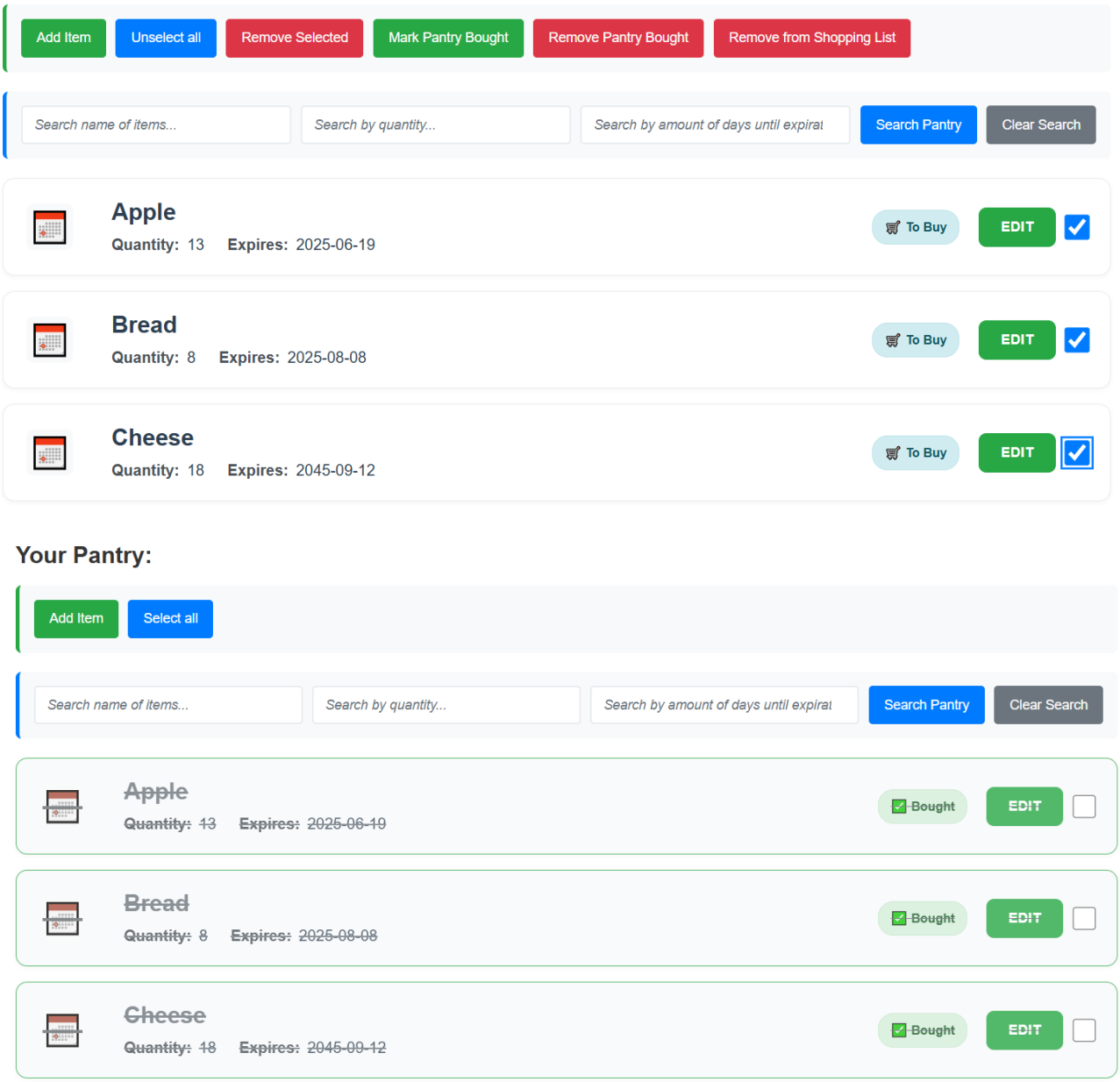

Pantry Manager
Home
Your Pantry
Your Shopping List
Settings
Login
Sign Up
Forgot Password
Continue to Forgot Password
Your Pantry:
Add Item
Select all
Remove Selected
Search Pantry
Clear Search
Add Item to Pantry
Item Name:
Bread
Quantity:
8
Expiration Date:
08/08/2025
Add Item


Your Pantry:
Add Item
Select all
Search name of items...
Search by quantity...
Search by amount of days until expirati
Search Pantry
Clear Search
Bread
Quantity: 8
Expires: 2025-08-08
EDIT


Your Pantry:
Add Item
Select all
Search name of items...
Search by quantity...
Search by amount of days until expirat
Search Pantry
Clear Search
Bread
Quantity: 8
Expires: 2025-08-08
EDIT
Cheese
Quantity: 18
Expires: 2045-09-12
EDIT
Apple
Quantity: 13
Expires: 2025-06-19
EDIT

Apple
Search by quantity...
Search by amount of days until expirati
Search Pantry
Clear Search
Apple
Quantity: 13
Expires: 2025-06-19
EDIT

Your Pantry:



Your Pantry:

Lessons learned for the project

Andrew Feran:

I learned a lot from this experience! To get more specific: I really feel that this project has given me a much better understanding of the requirements engineering process. Before coming into this I had somewhat of an understanding of this process through work/other courses but I feel that this project really helped me hone my technical writing. Additionally, I'm so glad to have finally gotten to do a full SDLC as well. Overall, I'm very happy with the outcome and would love to do more like this.

Arselle Conteh:

One of the main lessons that I learned from this project was the importance of planning and time management. The research process was a lot more in depth than I initially realized. I never realized how much went into the backend process of these types of systems. When creating the activity diagram, I was able to establish a flow state that shows how to properly implement these features. Using these diagrams helped me see a visual of what the final product should look like and how it should act. Also, the questions that we answered during the first few assignments painted a clear picture of how we're going to practically implement this, so it was nice to see that as well.

Benjamin Bruinsma:

The lessons I learned here in this project are paramount to my success as a developer. I was in charge of organizing the User Interface for our project. I spent a great amount of time in the design chair testing many icons, layouts, and colors. I created the design using online website design software and did research into similar software programs. For some of the more personalized designs, I used MS Paint to create a one of a kind system look. In the end, I wrote much of the HTML for the interaction between the computer and user. I also implemented the dark mode option for users who prefer to use the system at night.

Davi Borges:

It was interesting working with a more complete process than I had experienced in any previous project. The Software Requirements Specification and Software Design Specification made an interesting flow of work. Having to build from the beginning, without many instructions or requirements.

When working with the database, I was surprised by how smoothly things went, after I had everything already planned for the Software Design Specification. Having the schema, relationships, and even security considerations like password hashing and data validation detailed beforehand in the SDS, made the implementation phase significantly more efficient.

I've never worked on a project this complex before, mostly because of how many parts there are. Getting to know and understand each of them was a challenge. But because of that, I've learned about XAMPP and Apache, which made my work easier, and it was compatible with the front and back end.

Nickolas Diaz: (Your lessons)

Creating the logic of the system and tying the user interface, client logic, server logic and database logic was challenging. Developing most of the php backend and client logic make you really aware of how the whole system works. I really had to think about how the data between systems interact with each other. For example the user registration would go something like user presses sign up button, the browser would gather the email and password from the fields and send it to the php backend, from the backend the email is check if it does not exist or it is not verified from the database if so send an email with a verification code. If email is sent then create/update the user to the database and lastly respond to the user with a success equals true. Once the client receives the response, display the response to the user interface. The whole system has to really be tuned to account for all the possible edge cases, this means having lots of error messages, if statements and try catch statements to make sure the application is robust. The development was more like the agile framework as It was a cycle of creating a feature, testing it, finding bugs, and fixing the bugs. It was also interesting on how having layers of abstractions helped build the application, many functions that I created could be used in other places of logic. I also learned a lot about the full stack architecture and how to store, manipulate, test and deploy applications. Overall I am proud of what the team has achieved and what we built. I hope you enjoy our application. Pantry Management Website Pantry Management Source Code